

Intro to graphs

Discord + WhatsApp



Attendance

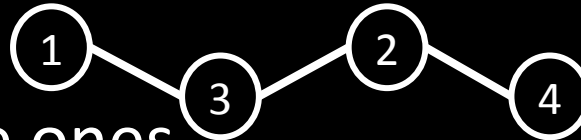


Definitions

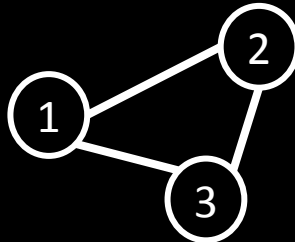
- Graph: $G = (V, E)$
- Set of vertices: $V = \{1, 2, 3, 4, 5, 6\}$
- Set of edges: $E = \{(1, 2), (1, 3), (2, 3), (4, 2), (6, 5)\}$

$$E = \{(1, 2), (1, 3), (2, 3), (4, 2), (6, 5)\}$$

- Path: series of vertices,
edge between consecutive ones

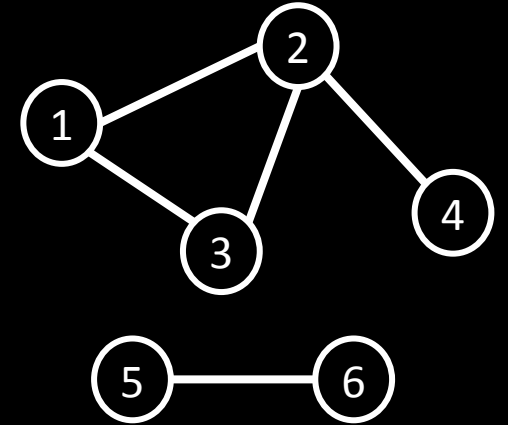


- Cycle: first and last one
is connected as well

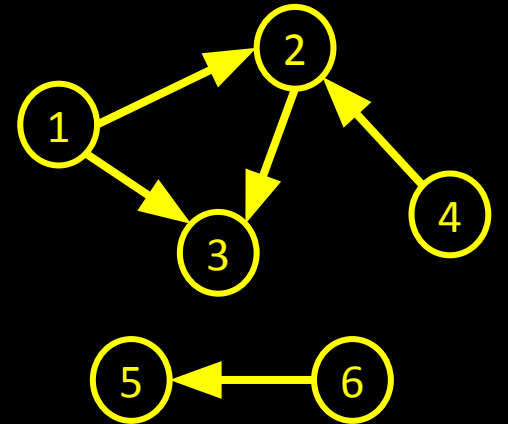


- Degree: the number of edges
connecting to a vertex

Undirected graph



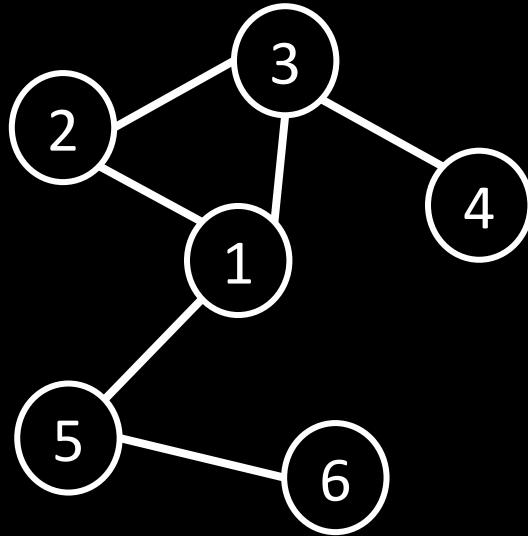
Directed graph



Storing graphs (graph representations)

Edge list

2 3
1 2
3 1
3 4
1 5
6 5



Adjacency list

1 : 2 3 4 5
2 : 3 1
3 : 2 1 4
4 : 3
5 : 1 6
6 : 5

Storing graphs (graph representations)

Input

6 6
2 3
1 2
3 1
3 4
1 5
6 5

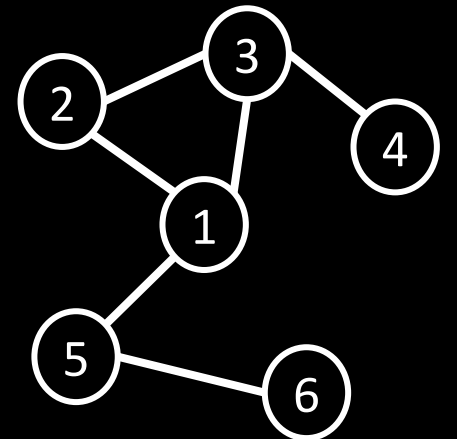
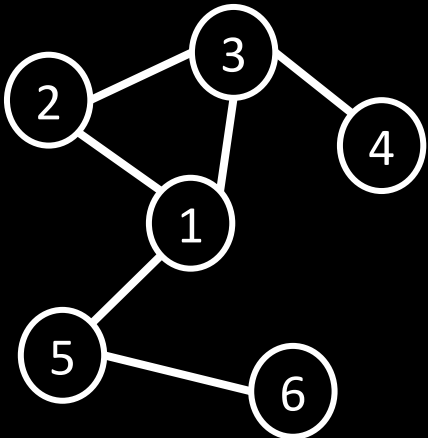
```
const int MAXN = 100001;
vector<int> g[MAXN];

int main() {
    int n, m;
    cin >> n >> m;
    for (int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;
        g[u].push_back(v);
        g[v].push_back(u); // undirected
    }

    for (int i = 1; i <= n; i++) {
        cout << i << ": ";
        for (int v : g[i]) cout << v << " ";
        cout << endl;
    }
}
```

Storing

1: 2 3 5
2: 3 1
3: 2 1 4
4: 3
5: 1 6
6: 5



Storing graphs (graph representations)

Edge list

2 3

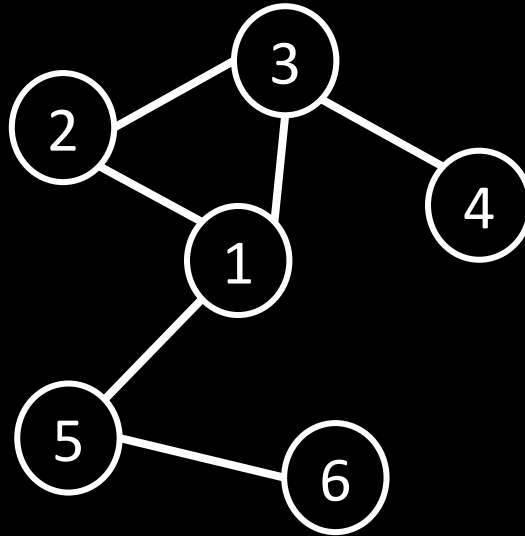
1 2

3 1

3 4

1 5

6 5



Adjacency matrix

	1	2	3	4	5	6
1		1	1		1	
2	1		1			
3	1	1		1		
4			1			
5	1					1
6					1	

Storing graphs (graph representations)

Input

6 6

2 3

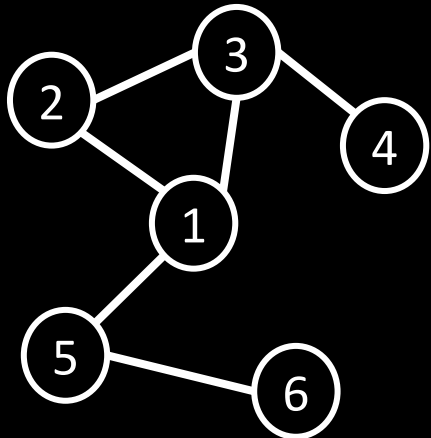
1 2

3 1

3 4

1 5

6 5



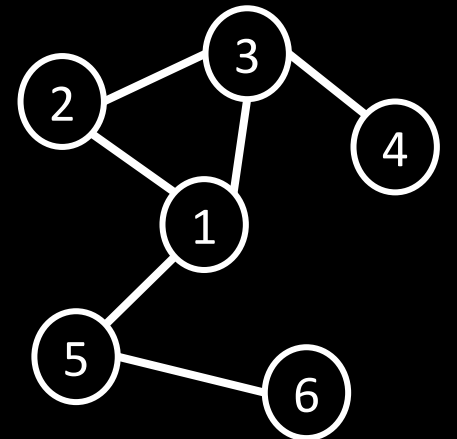
```
const int MAXN = 1001;
bool g[MAXN][MAXN];

int main() {
    int n, m;
    cin >> n >> m;
    for (int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;
        g[u][v] = true;
        g[v][u] = true; // undirected
    }

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++)
            cout << g[i][j] << " ";
        cout << endl;
    }
}
```

Storing

0	1	1	0	1	0
1	0	1	0	0	0
1	1	0	1	0	0
0	0	1	0	0	0
1	0	0	0	0	1
0	0	0	0	1	0



Comparison - memory

N = no of vertices, E = no of edges

N = 1000
E = 1000

N = 1000
E = 100 000

N = 5000
E = 200 000

N = 100 000
E = 200 000

- Adjacency list: $O(E)$

- Approximately as much memory as the input

~ 0.01 MB

~ 1 MB

~ 2 MB

~ **2 MB**

- Adjacency matrix: $O(N^2)$

- Wasteful in case of sparse graphs
- In practice: $N \leq 5000$

~ 1 MB

~ 1 MB

~ 25 MB

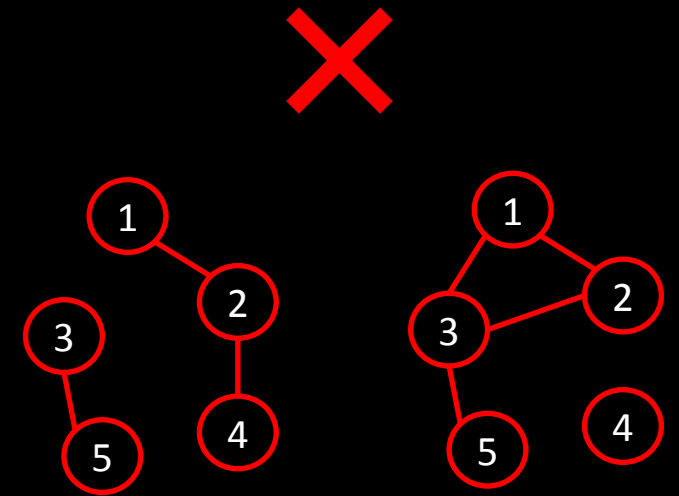
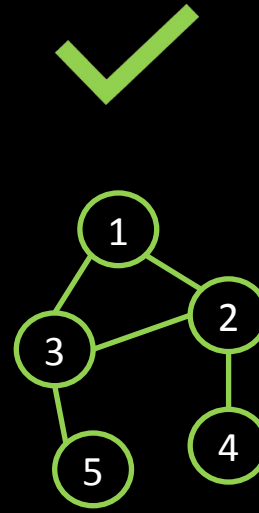
~ **10 000 MB**

Comparison - operations

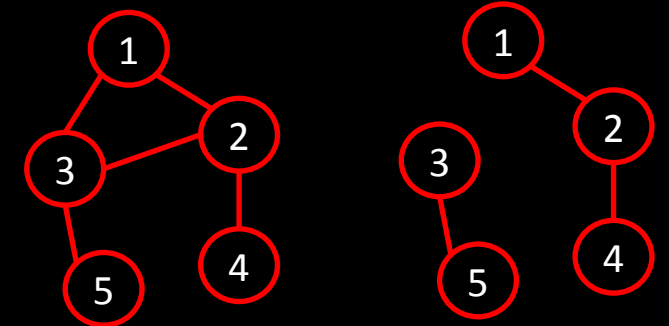
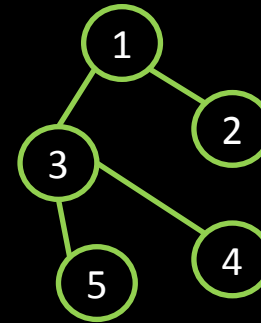
	Adjacency list	Matrix	Edge list
Does (u,v) exist?	$O(\text{degree})$ iterate through $g[u]$	$O(1)$ $g[u][v]$	$O(E)$ look at all edges
Neighbours of u	$O(\text{degree})$ iterate through $g[u]$	$O(N)$ iterate through $g[u][1..N]$	$O(E)$ look at all edges
Inspect all edges	$O(E)$	$O(N^2)$	$O(E)$
Add edge	$O(1)$	$O(1)$	$O(1)$
Delete edge	$O(\text{degree})$	$O(1)$	$O(E)$
Add vertex	$O(1)$	$O(N)$	$O(1)$
Delete vertex	??	??	$O(E)$

Some more definitions

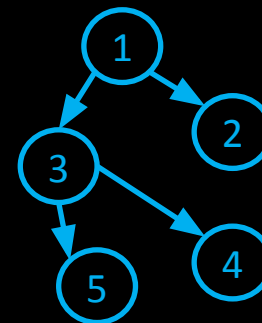
- Connected graph:
a path exists between any two vertices
 - *For directed graphs (strong connectivity)*



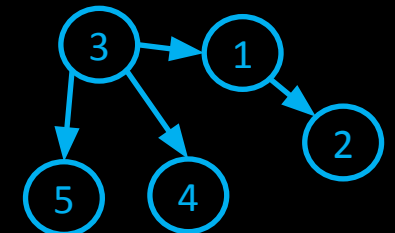
- Tree graph:
Acyclic connected graph
 - *Remark: in a tree there is exactly one path between any two vertices*



- Rooted tree:
there is a special root node
 - *Edges: parent $\rightarrow$ child relationships*



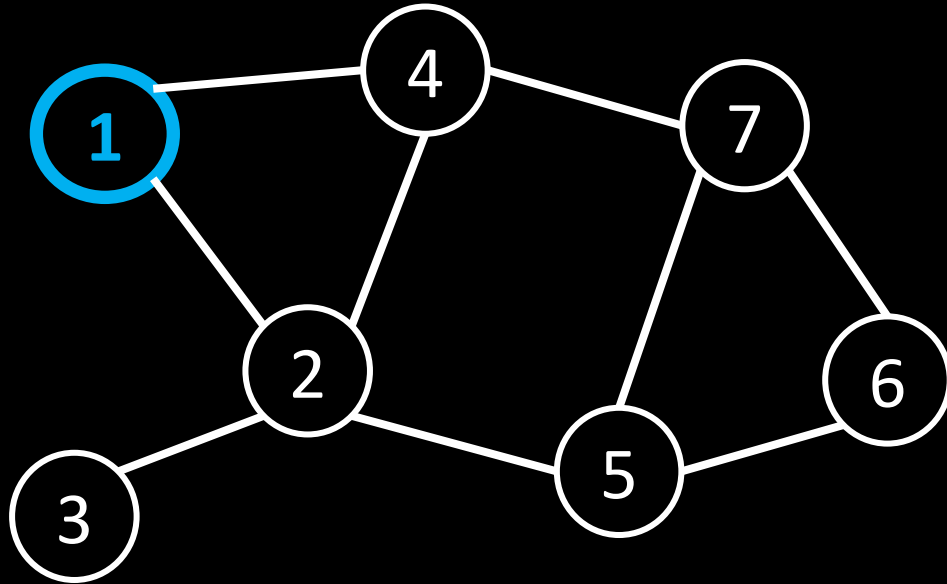
Same
tree with
different
root:



Graph traversals

Iterate through the vertices in some order along the edges

Breadth-first search



1

Adjacency list:

1 : 4 2

2 : 1 3 5 4

3 : 2

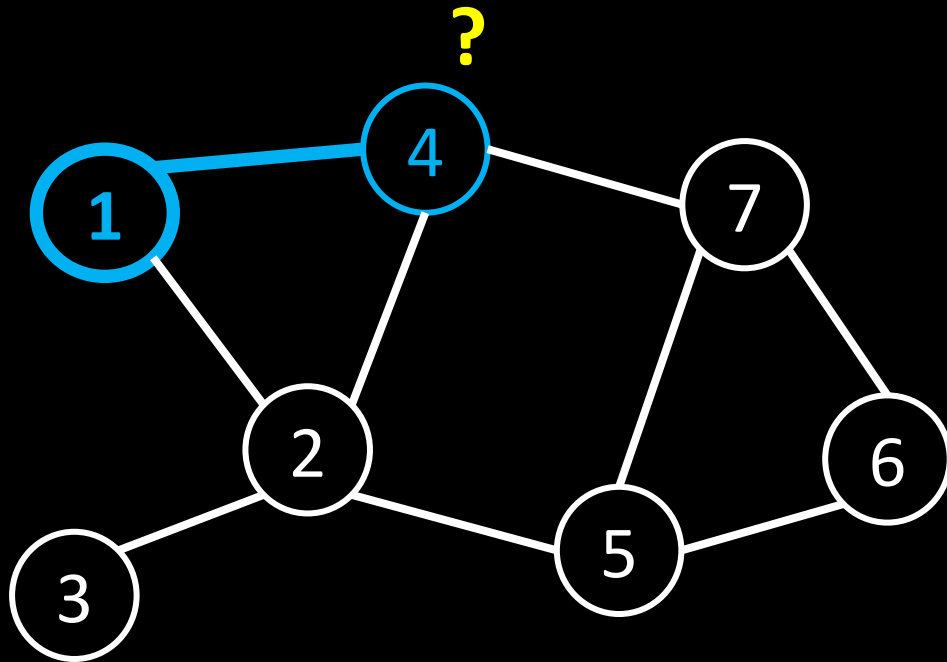
4 : 1 2 7

5 : 2 7 6

6 : 7 5

7 : 4 5 6

Breadth-first search

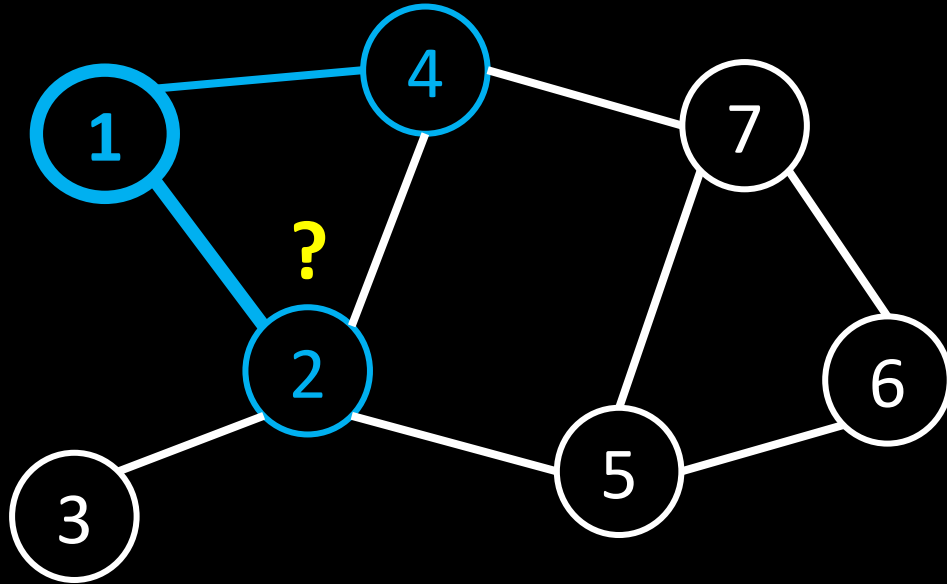


1 4

Adjacency list:

1: 4 2
2: 1 3 5 4
3: 2
4: 1 2 7
5: 2 7 6
6: 7 5
7: 4 5 6

Breadth-first search

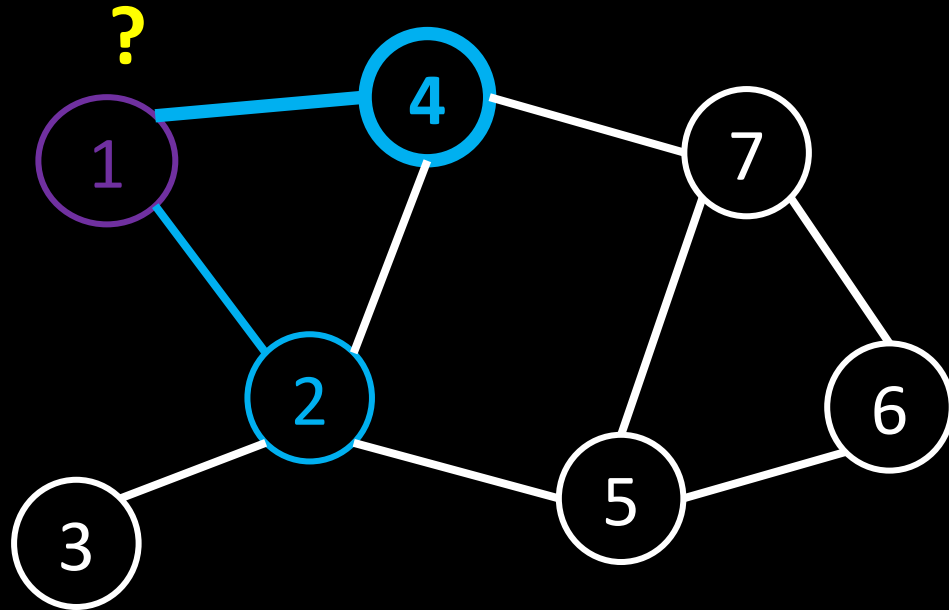


1 4 2

Adjacency list:

```
1: 4 2
2: 1 3 5 4
3: 2
4: 1 2 7
5: 2 7 6
6: 7 5
7: 4 5 6
```

Breadth-first search



1 4 2

Adjacency list:

1: 4 2

2: 1 3 5 4

3: 2

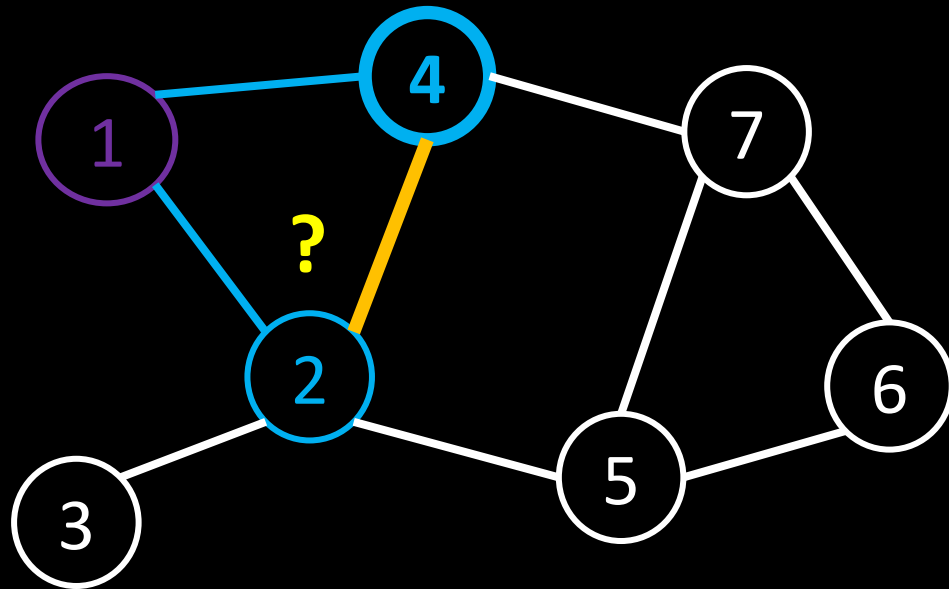
4: 1 2 7

5: 2 7 6

6: 7 5

7: 4 5 6

Breadth-first search



1 4 2

Adjacency list:

1: 4 2

2: 1 3 5 4

3: 2

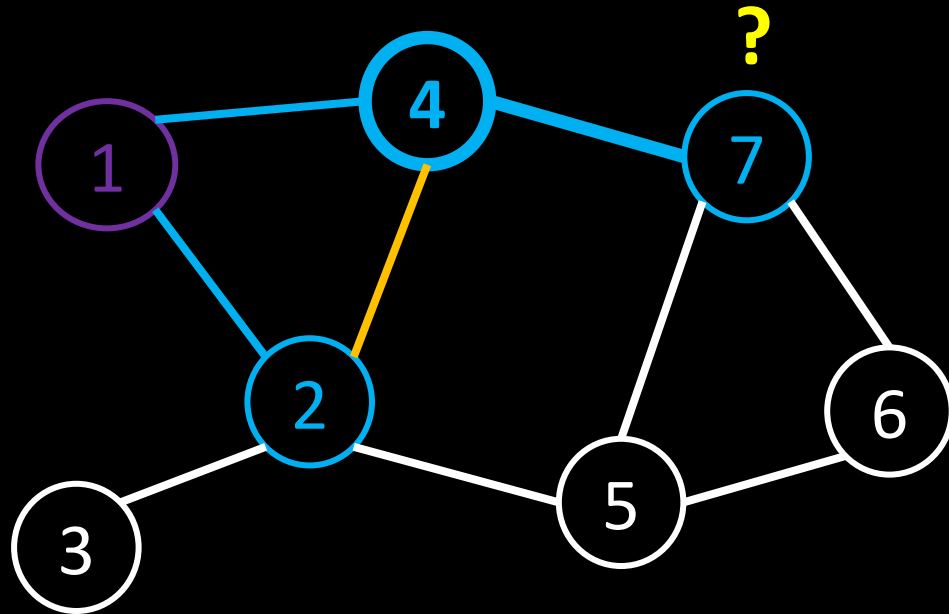
4: 1 2 7

5: 2 7 6

6: 7 5

7: 4 5 6

Breadth-first search



1 4 2 7

Adjacency list:

1: 4 2

2: 1 3 5 4

3: 2

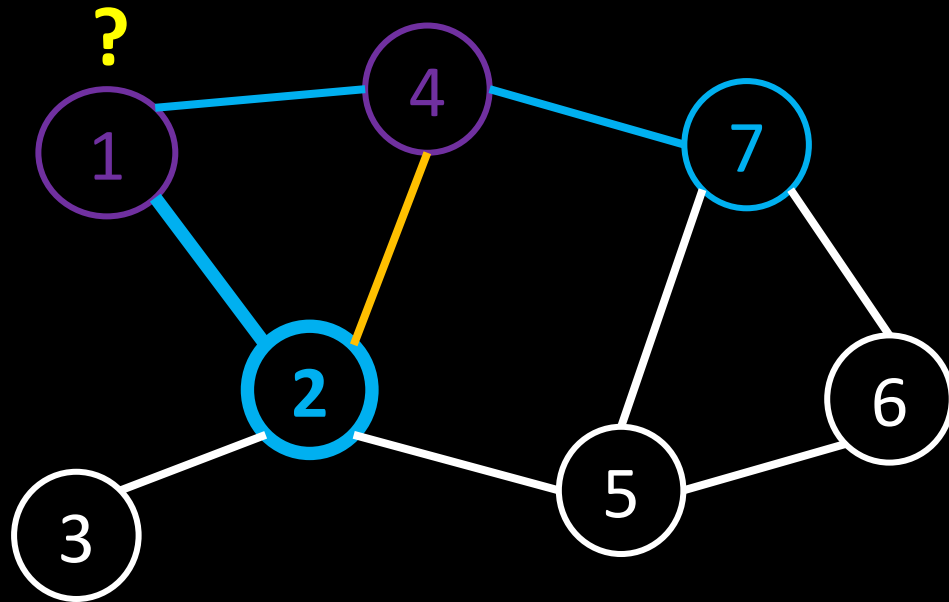
4: 1 2 7

5: 2 7 6

6: 7 5

7: 4 5 6

Breadth-first search



1 4 2 7

Adjacency list:

1: 4 2

2: 1 3 5 4

3: 2

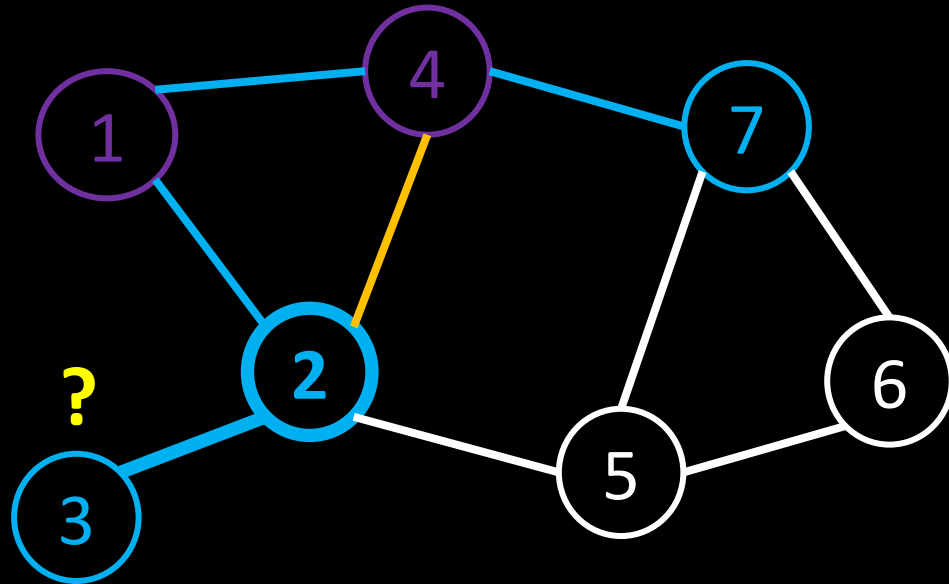
4: 1 2 7

5: 2 7 6

6: 7 5

7: 4 5 6

Breadth-first search



1 4 2 7 3

Adjacency list:

1 : 4 2

2 : 1 3 5 4

3 : 2

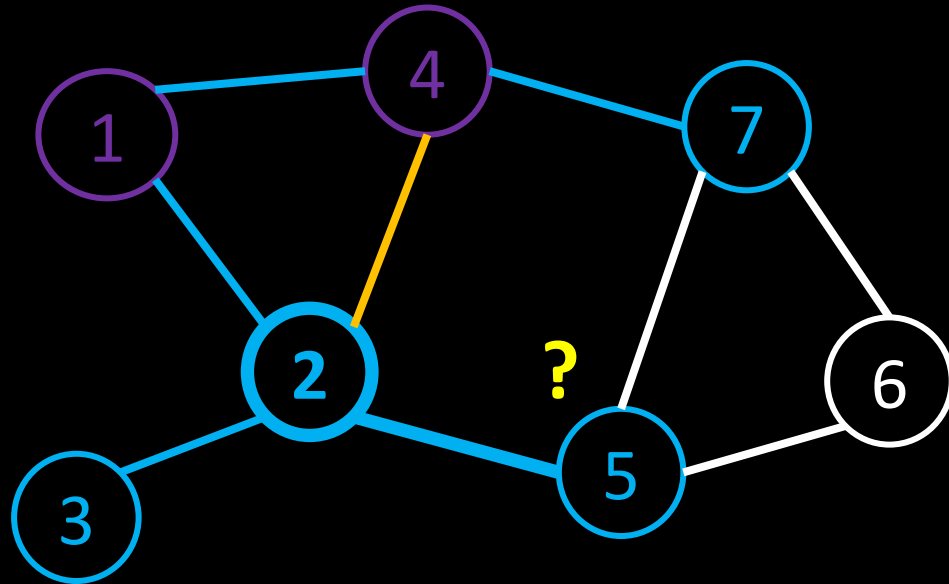
4 : 1 2 7

5 : 2 7 6

6 : 7 5

7 : 4 5 6

Breadth-first search



1 4 2 7 3 5

Adjacency list:

1: 4 2

2: 1 3 5 4

3: 2

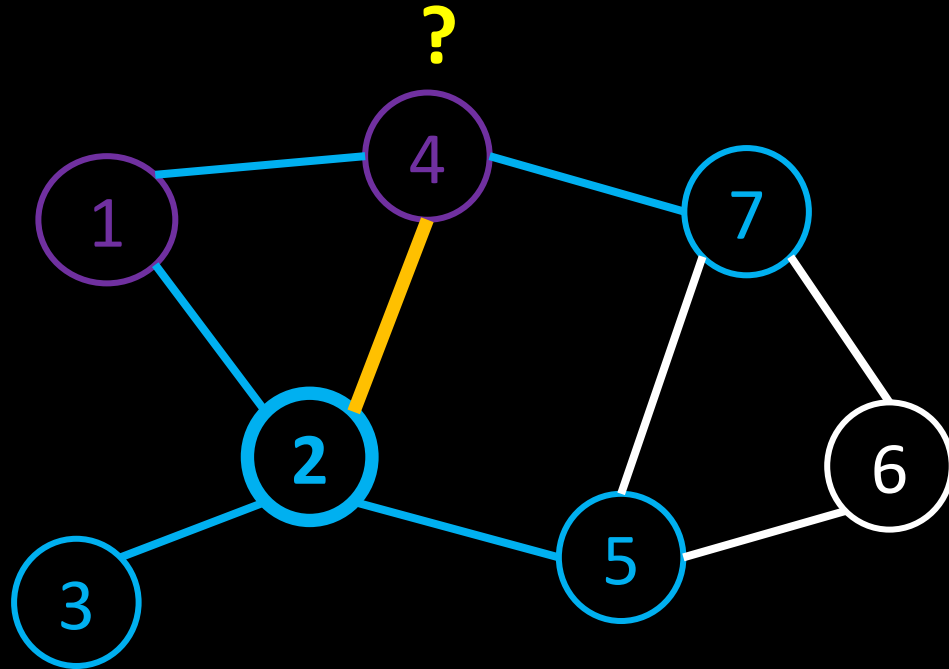
4: 1 2 7

5: 2 7 6

6: 7 5

7: 4 5 6

Breadth-first search



1 4 2 7 3 5

Adjacency list:

1: 4 2

2: 1 3 5 4

3: 2

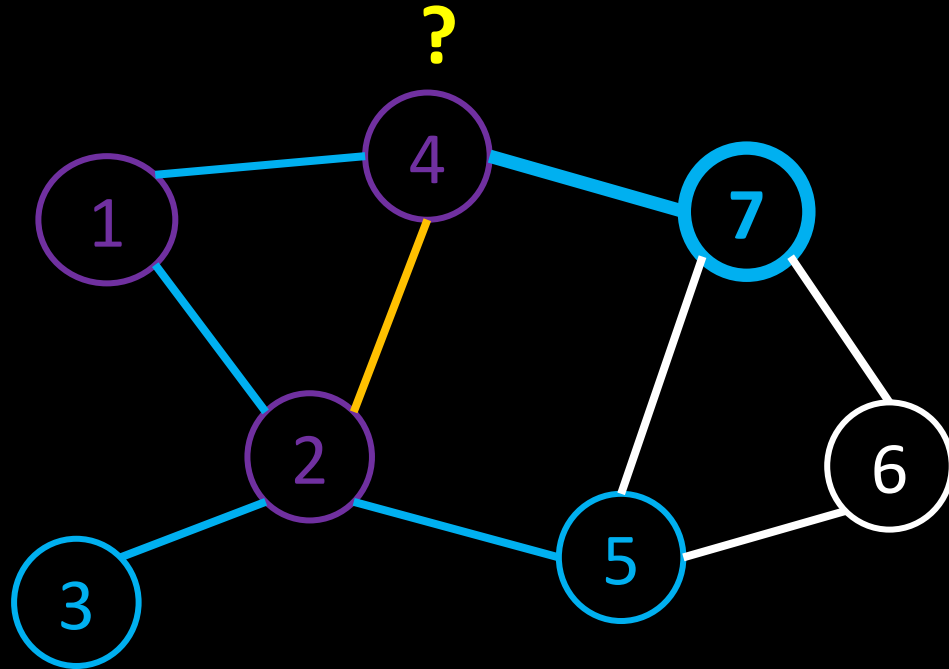
4: 1 2 7

5: 2 7 6

6: 7 5

7: 4 5 6

Breadth-first search



1 4 2 7 3 5

Adjacency list:

1: 4 2

2: 1 3 5 4

3: 2

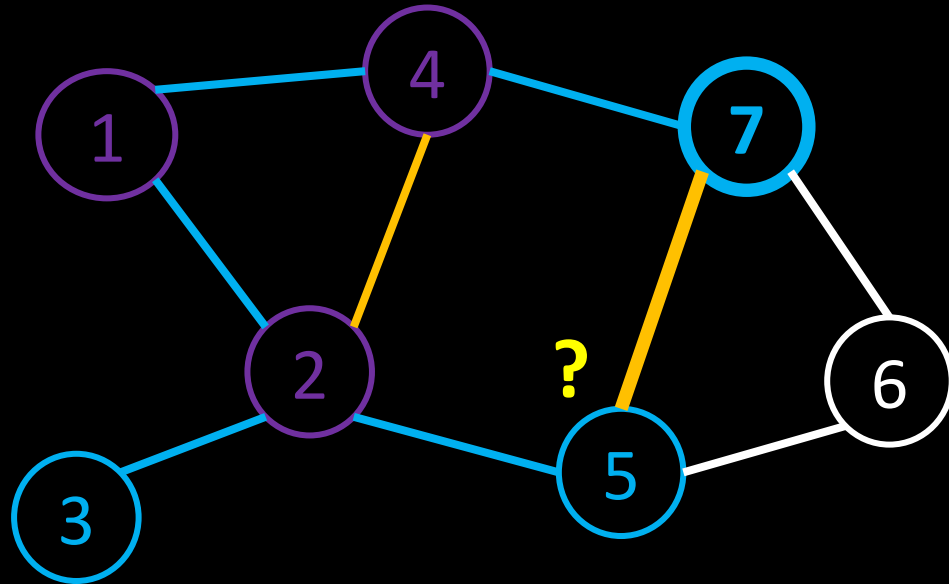
4: 1 2 7

5: 2 7 6

6: 7 5

7: 4 5 6

Breadth-first search



1 4 2 7 3 5

Adjacency list:

1: 4 2

2: 1 3 5 4

3: 2

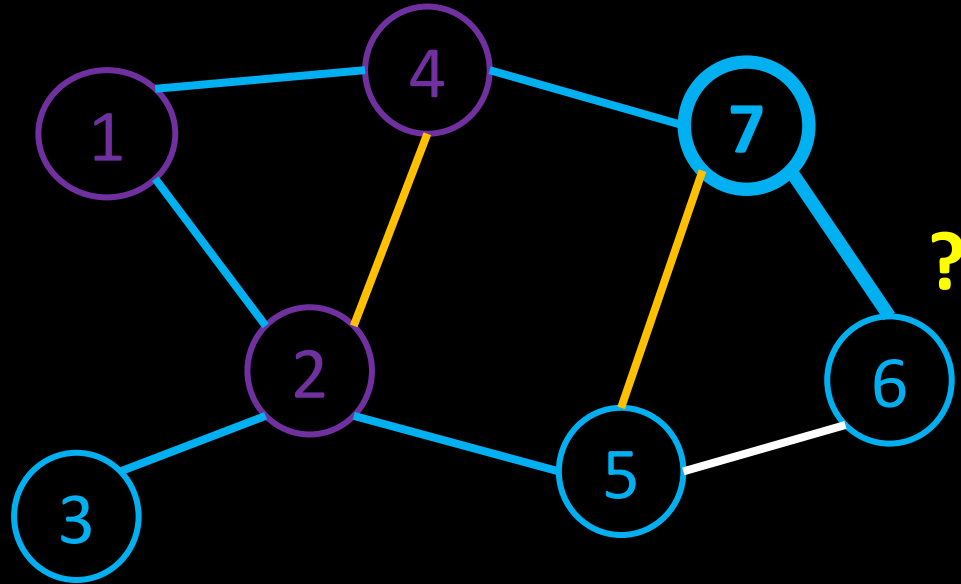
4: 1 2 7

5: 2 7 6

6: 7 5

7: 4 5 6

Breadth-first search



1 4 2 7 3 5 6

Adjacency list:

1: 4 2

2: 1 3 5 4

3: 2

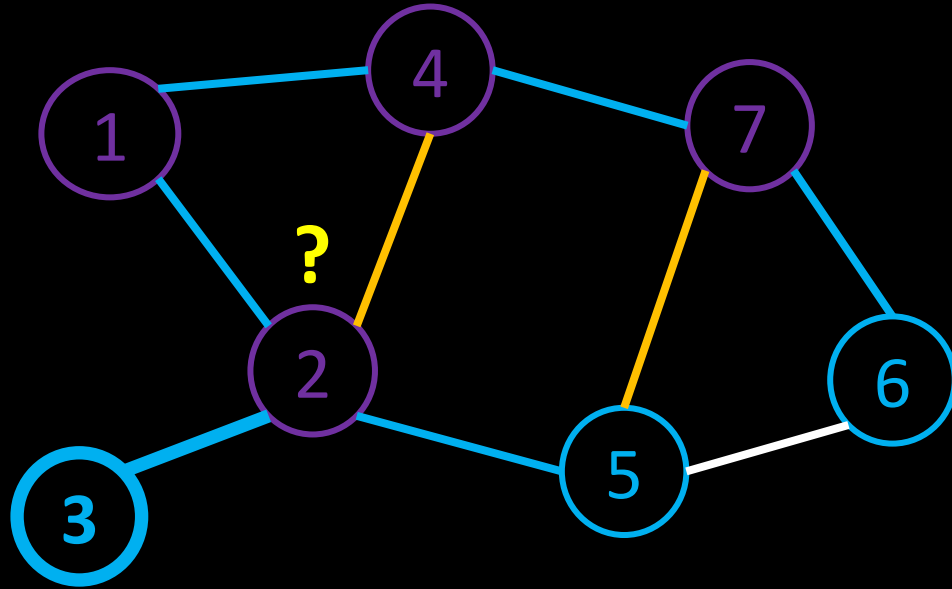
4: 1 2 7

5: 2 7 6

6: 7 5

7: 4 5 6

Breadth-first search



1 4 2 7 3 5 6

Adjacency list:

1: 4 2

2: 1 3 5 4

3: 2

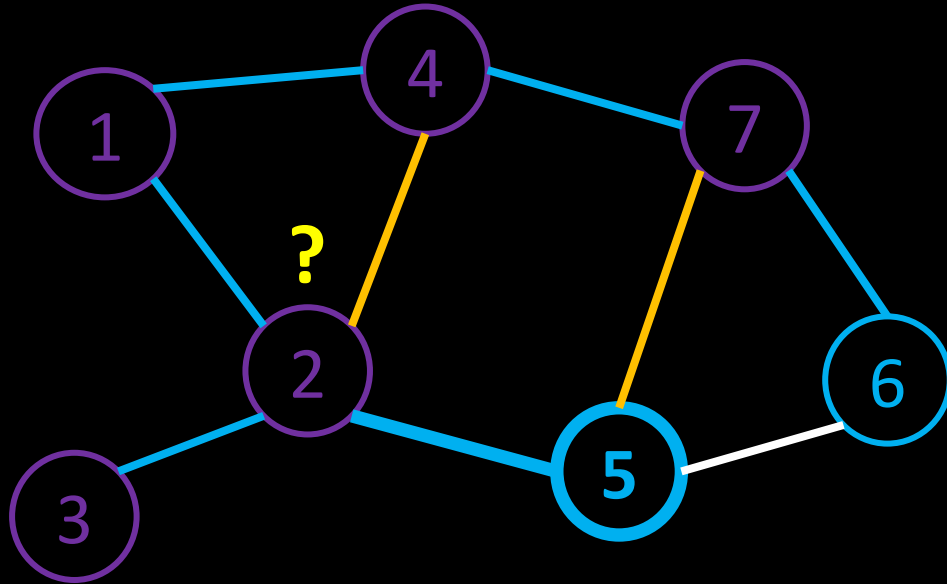
4: 1 2 7

5: 2 7 6

6: 7 5

7: 4 5 6

Breadth-first search



1 4 2 7 3 5 6

Adjacency list:

1: 4 2

2: 1 3 5 4

3: 2

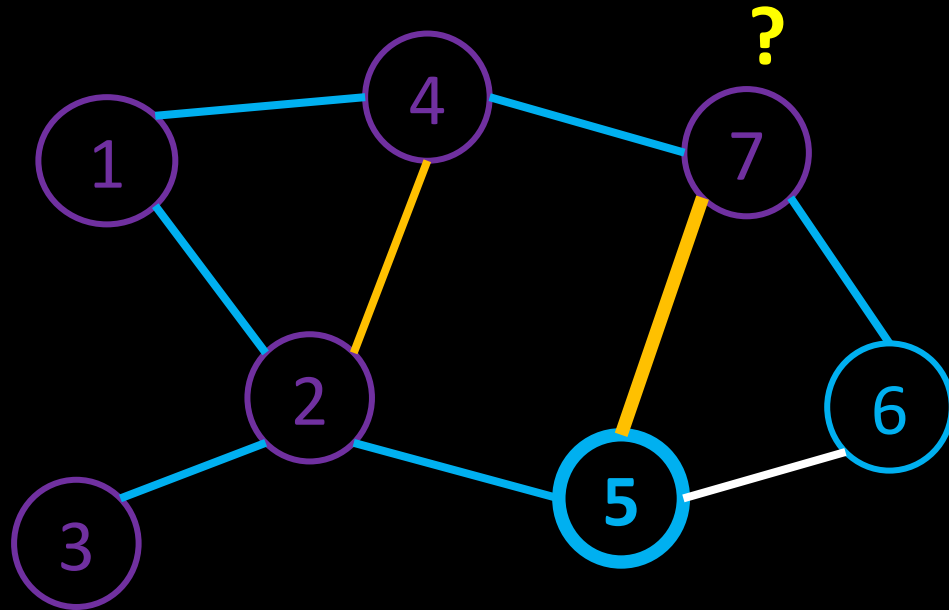
4: 1 2 7

5: 2 7 6

6: 7 5

7: 4 5 6

Breadth-first search



1 4 2 7 3 5 6

Adjacency list:

1: 4 2

2: 1 3 5 4

3: 2

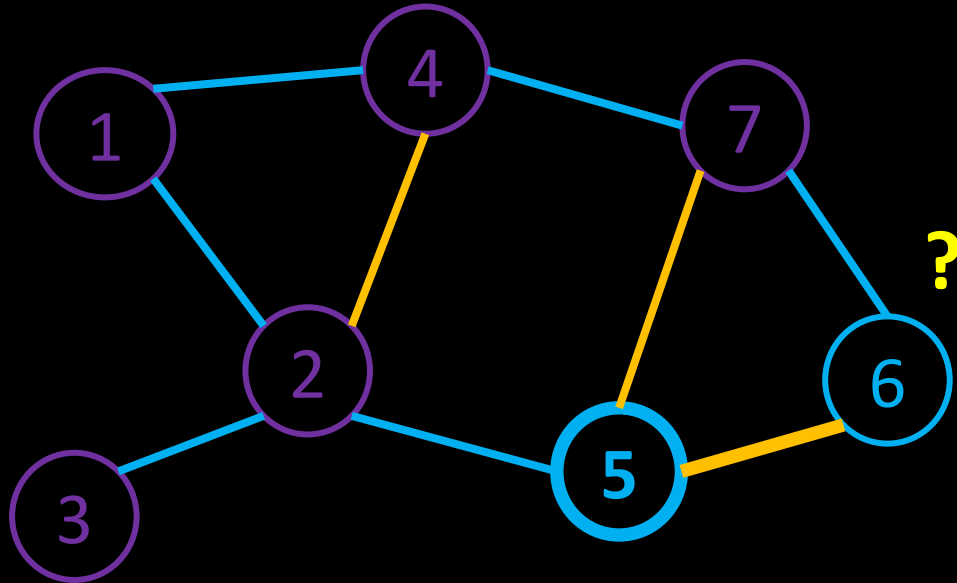
4: 1 2 7

5: 2 7 6

6: 7 5

7: 4 5 6

Breadth-first search



1 4 2 7 3 5 6

Adjacency list:

1: 4 2

2: 1 3 5 4

3: 2

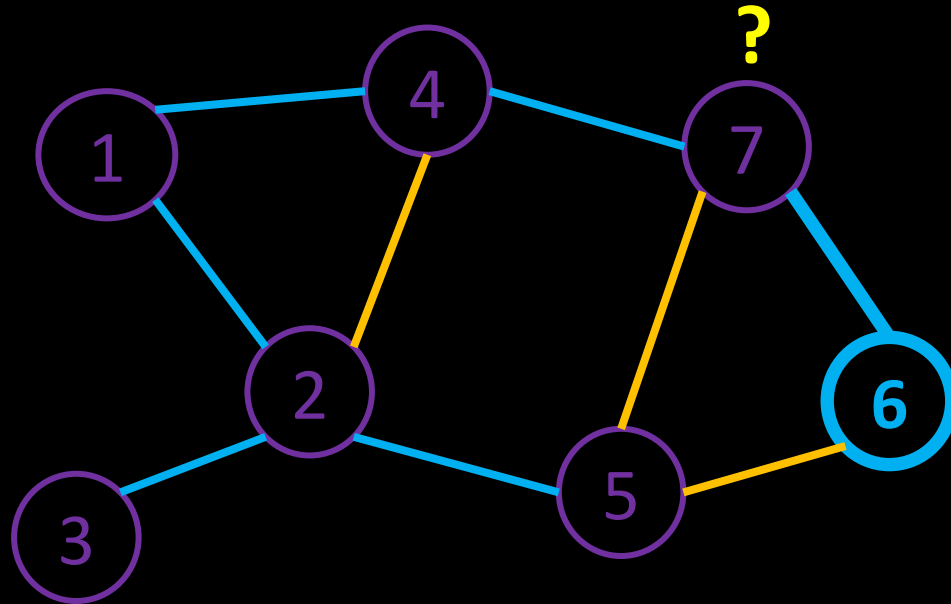
4: 1 2 7

5: 2 7 6

6: 7 5

7: 4 5 6

Breadth-first search



1 4 2 7 3 5 6

Adjacency list:

1: 4 2

2: 1 3 5 4

3: 2

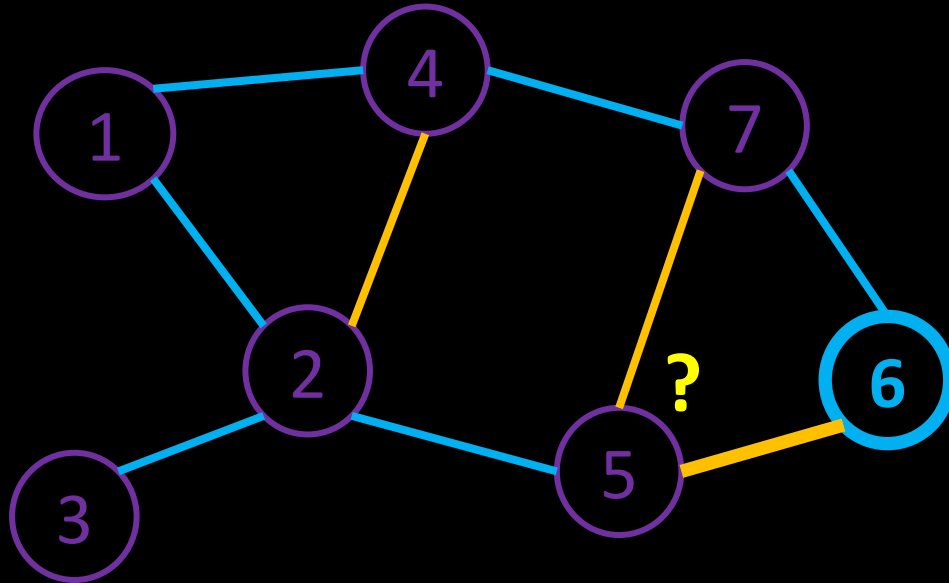
4: 1 2 7

5: 2 7 6

6: 7 5

7: 4 5 6

Breadth-first search



1 4 2 7 3 5 6

Adjacency list:

1: 4 2

2: 1 3 5 4

3: 2

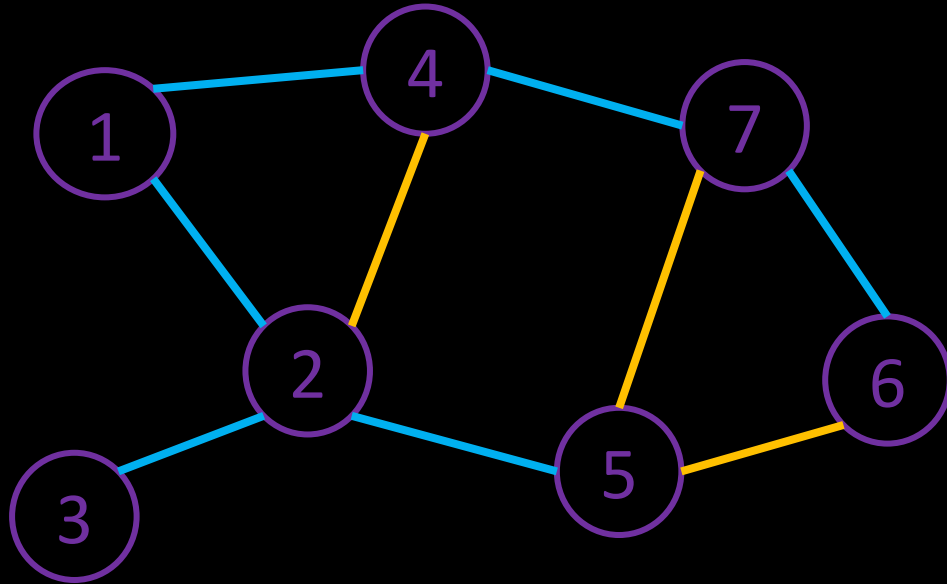
4: 1 2 7

5: 2 7 6

6: 7 5

7: 4 5 6

Breadth-first search



1 4 2 7 3 5 6

Adjacency list:

1: 4 2

2: 1 3 5 4

3: 2

4: 1 2 7

5: 2 7 6

6: 7 5

7: 4 5 6

Breadth-first search

```
vector<int> g[MAXN];  
bool visited[MAXN];
```

Breadth-First Search

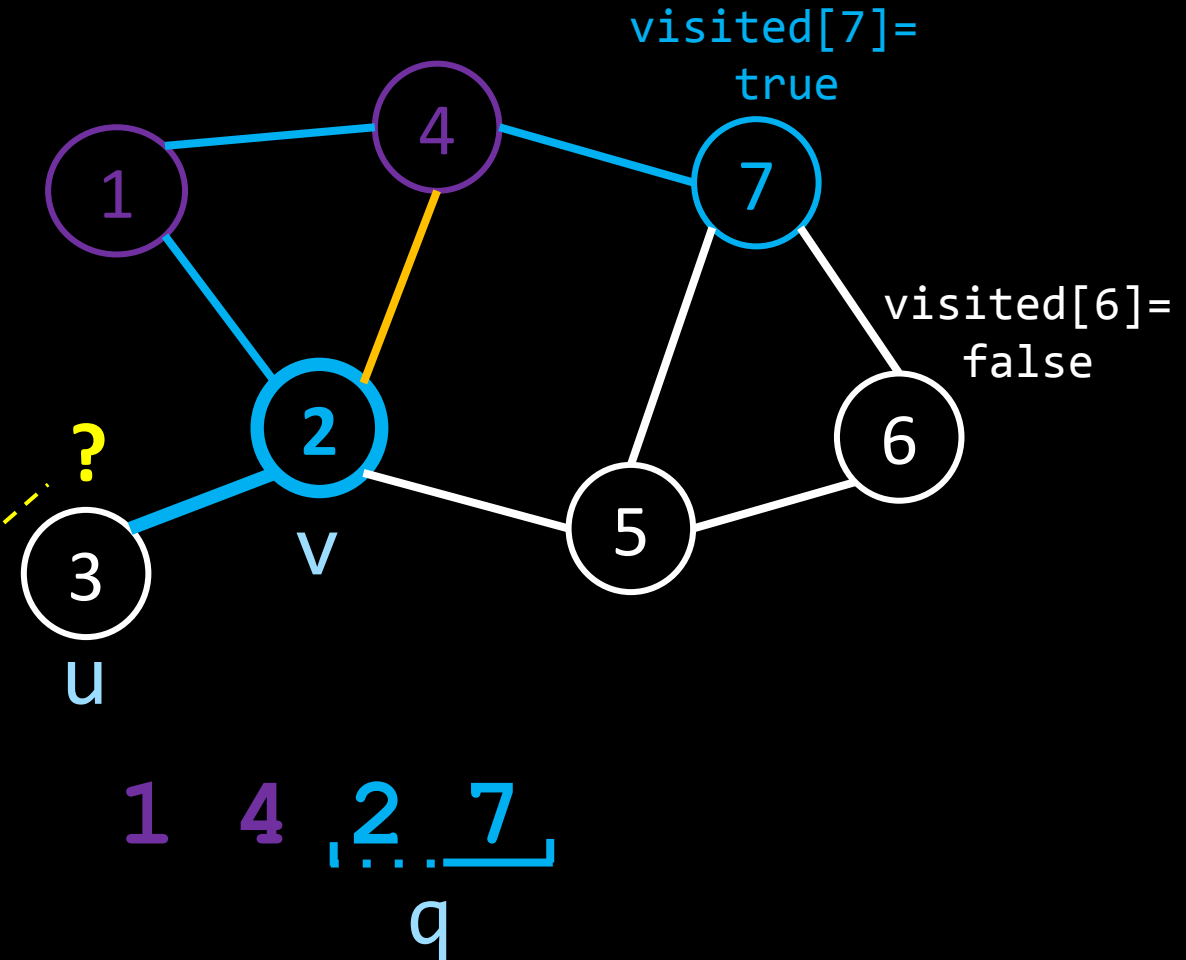
```
void bfs(int start) {  
    queue<int> q;  
    q.push(start);  
    visited[start] = true;  
    while (!q.empty()) {  
        int v = q.front();  
        q.pop();  
        for (int u : g[v]) {  
            if (!visited[u]) {  
                q.push(u);  
                visited[u] = true;  
            }  
        }  
    }  
}
```

Breadth-first search

```
vector<int> g[MAXN];  
bool visited[MAXN];
```

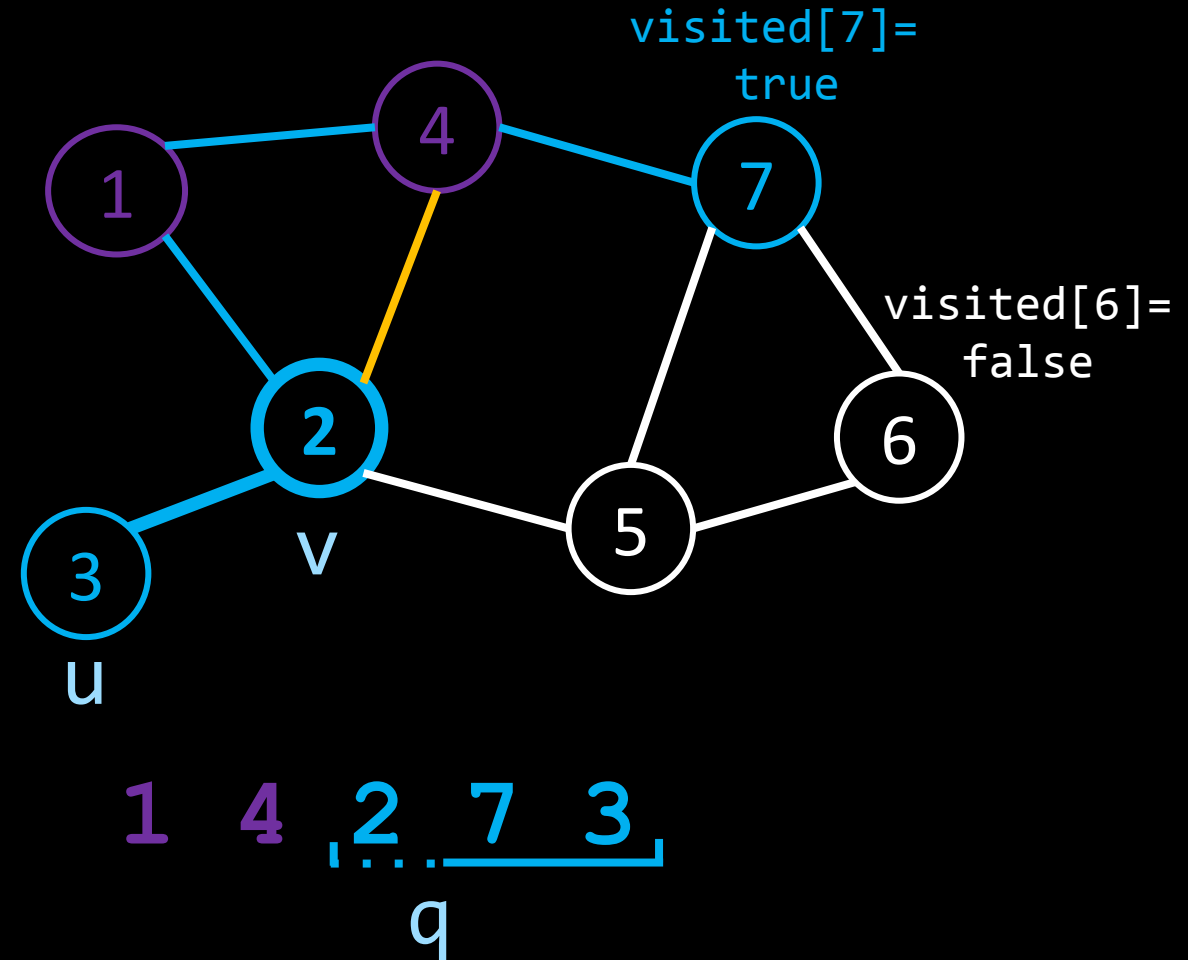
```
void bfs(int start) {  
    queue<int> q;  
    q.push(start);  
    visited[start] = true;  
    while (!q.empty()) {  
        int v = q.front();  
        q.pop();  
        for (int u : g[v]) {
```

```
            if (!visited[u]) {  
                q.push(u);  
                visited[u] = true;  
            }  
        }  
    }  
}
```



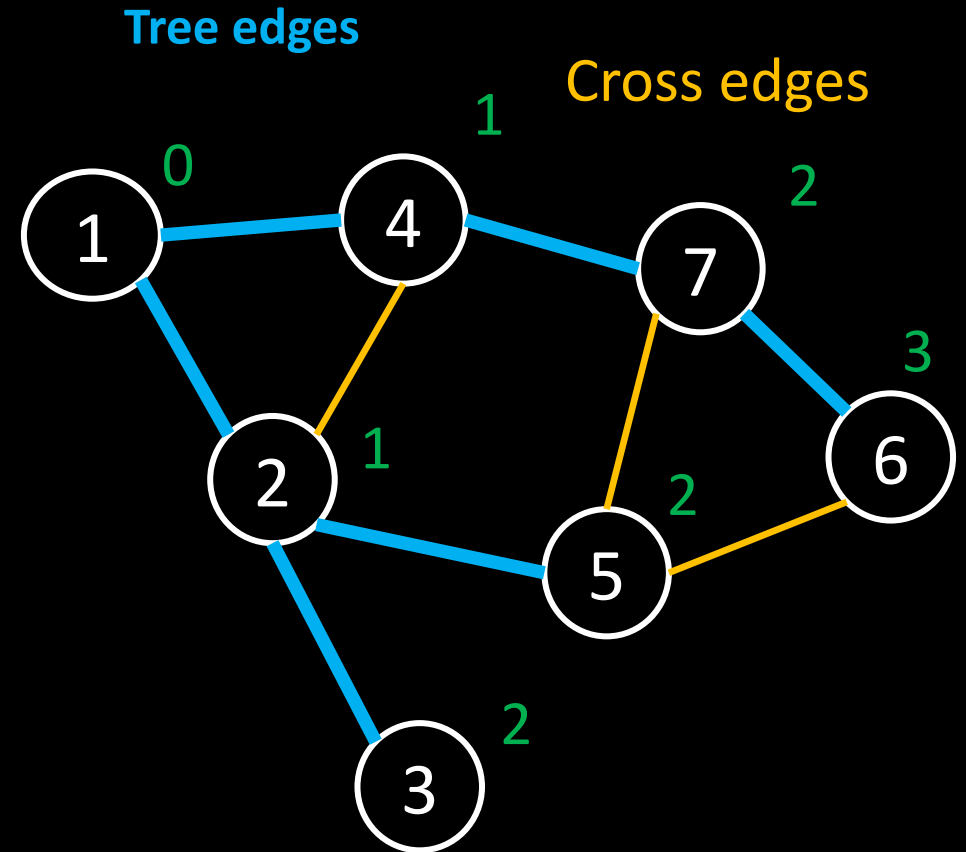
Breadth-first search

```
vector<int> g[MAXN];  
bool visited[MAXN];  
  
void bfs(int start) {  
    queue<int> q;  
    q.push(start);  
    visited[start] = true;  
    while (!q.empty()) {  
        int v = q.front();  
        q.pop();  
        for (int u : g[v]) {  
            if (!visited[u]) {  
                q.push(u);  
                visited[u] = true;  
            }  
        }  
    }  
}
```



Breadth-first search- properties

- The traversed edges create a tree
- Traverses the edges in increasing order of distance from the starting node
- The tree edges give the shortest path from the starting node to any other node
- Time complexity: $O(N + E)$



BFS - again

```
vector<int> g[MAXN];  
int parent[MAXN], dist[MAXN];
```

```
void bfs(int start) {  
    queue<int> q;  
    q.push(start);  
    dist[start] = 0;  
    parent[start] = start;
```

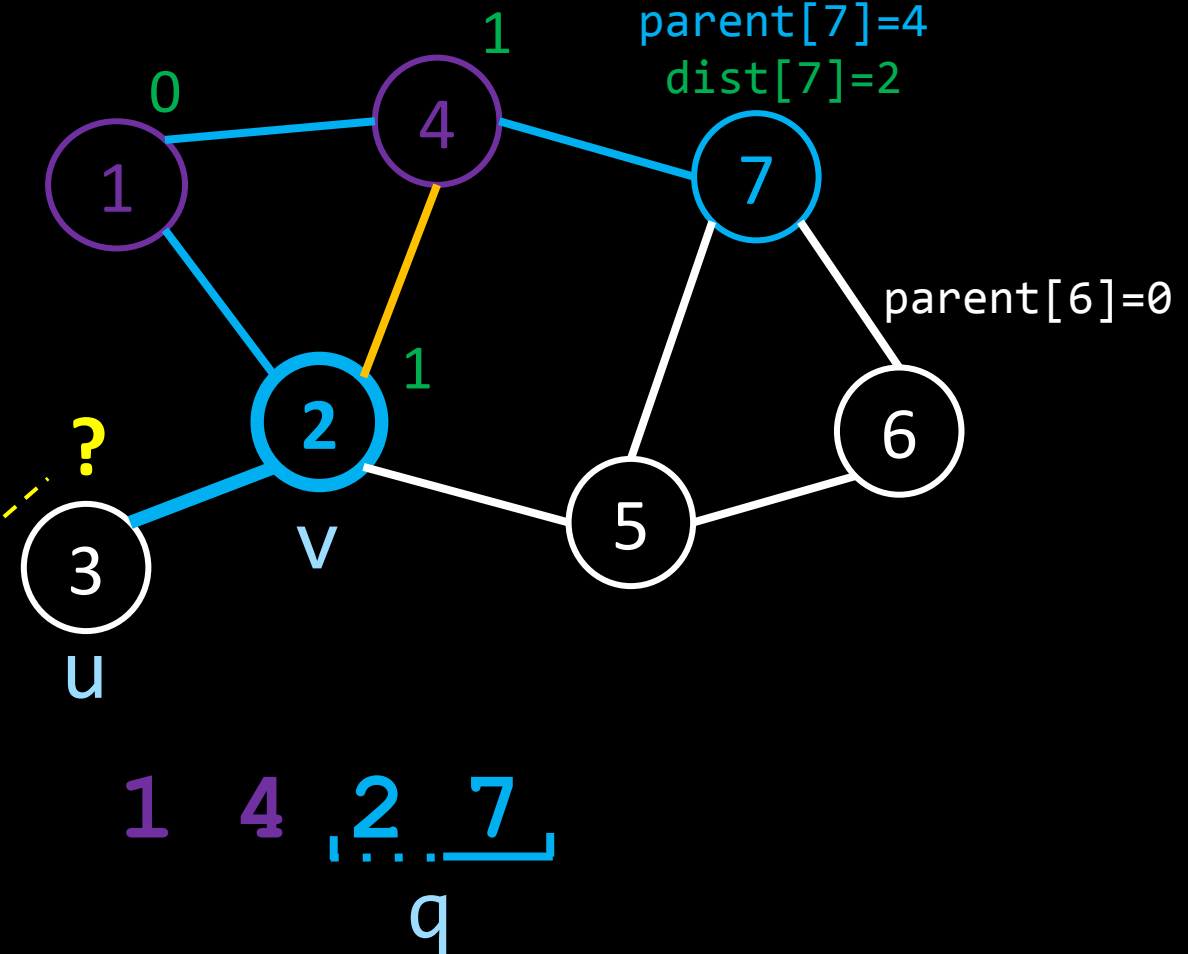
```
    while (!q.empty()) {  
        int v = q.front();  
        q.pop();  
        for (int u : g[v]) {
```

```
            if (parent[u] == 0) {  
                q.push(u);  
                parent[u] = v;  
                dist[u] = dist[v] + 1;  
            }
```

```
        }
```

```
    }
```

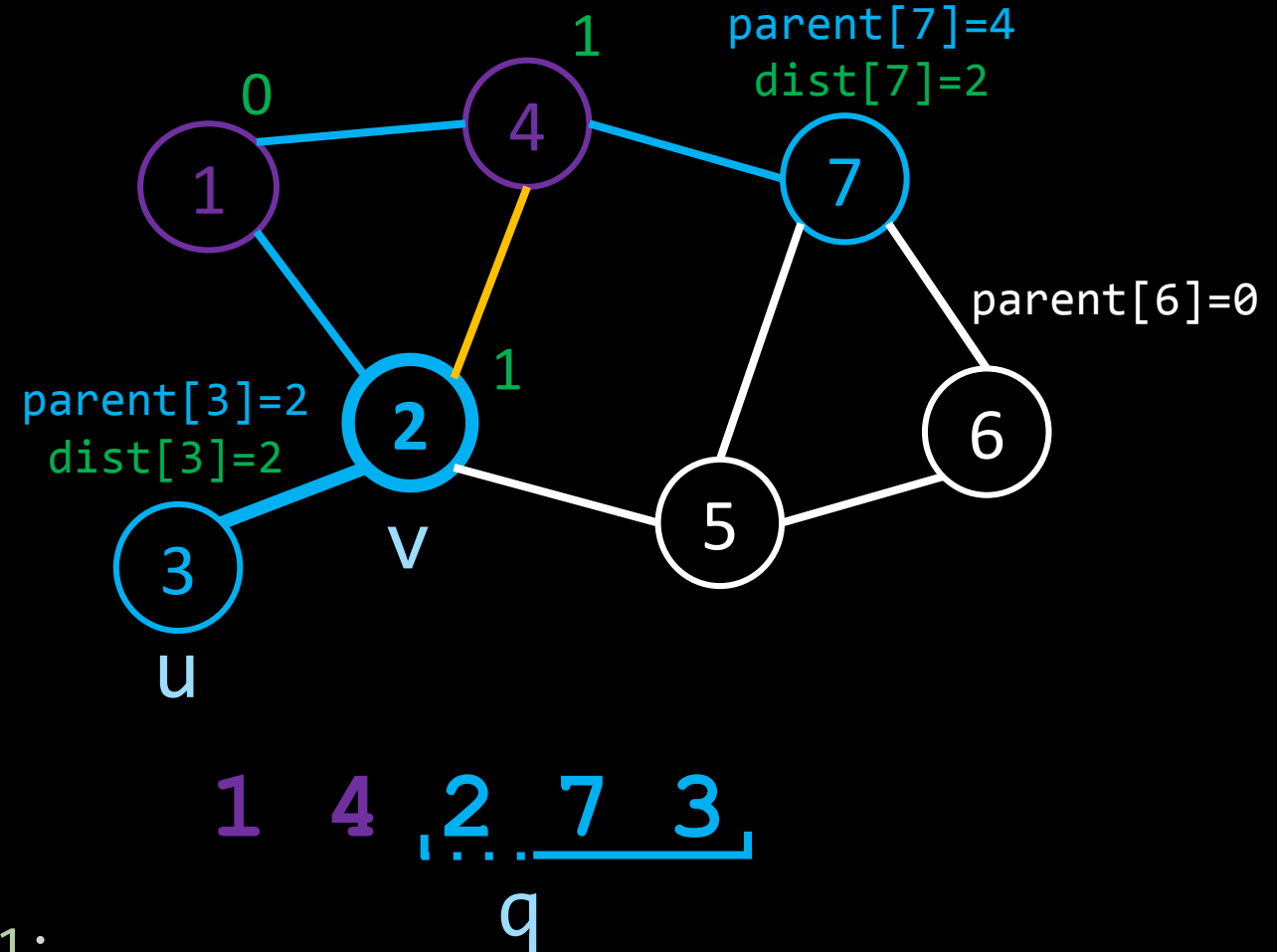
```
}
```



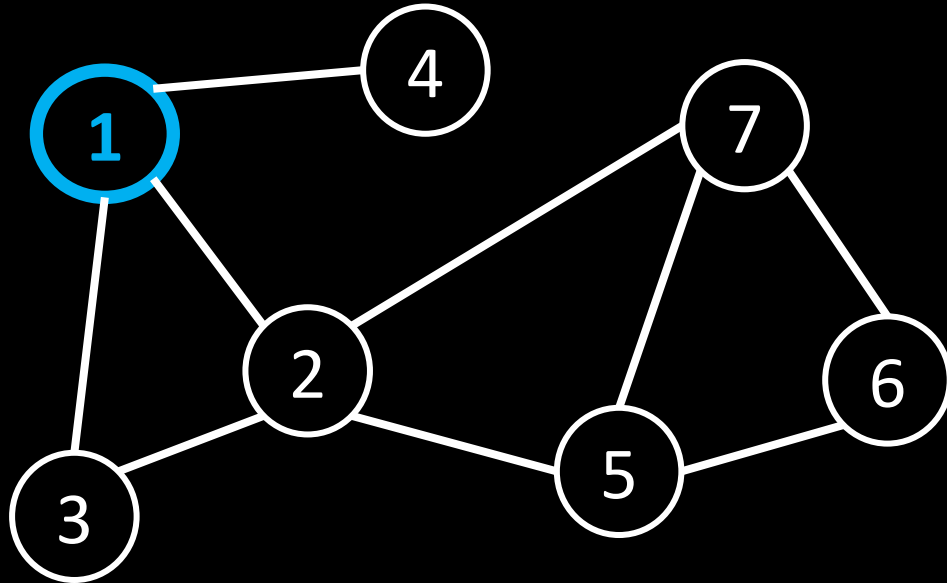
BFS - again

```
vector<int> g[MAXN];
int parent[MAXN], dist[MAXN];

void bfs(int start) {
    queue<int> q;
    q.push(start);
    dist[start] = 0;
    parent[start] = start;
    while (!q.empty()) {
        int v = q.front();
        q.pop();
        for (int u : g[v]) {
            if (parent[u] == 0) {
                q.push(u);
                parent[u] = v;
                dist[u] = dist[v] + 1;
            }
        }
    }
}
```



Depth-first search



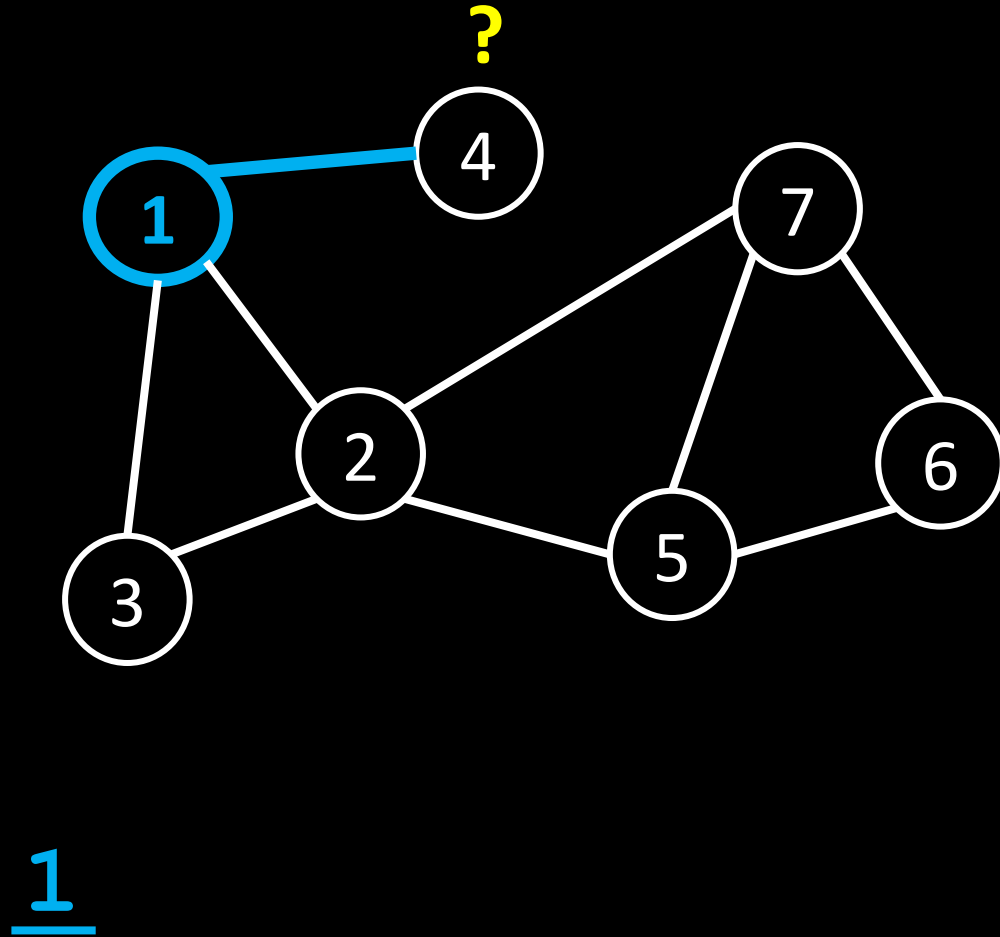
1

```
void dfs(int v) {  
    vis[v] = true;  
    for (int u : g[v])  
        if (!vis[u])  
            dfs(u);  
}
```

Adjacency list:

<u>1</u> :	4	2	3	dfs(1)
2:	1	3	5	7
3:	2	1		
4:	1			
5:	2	7	6	
6:	7	5		
7:	2	5	6	

Depth-first search

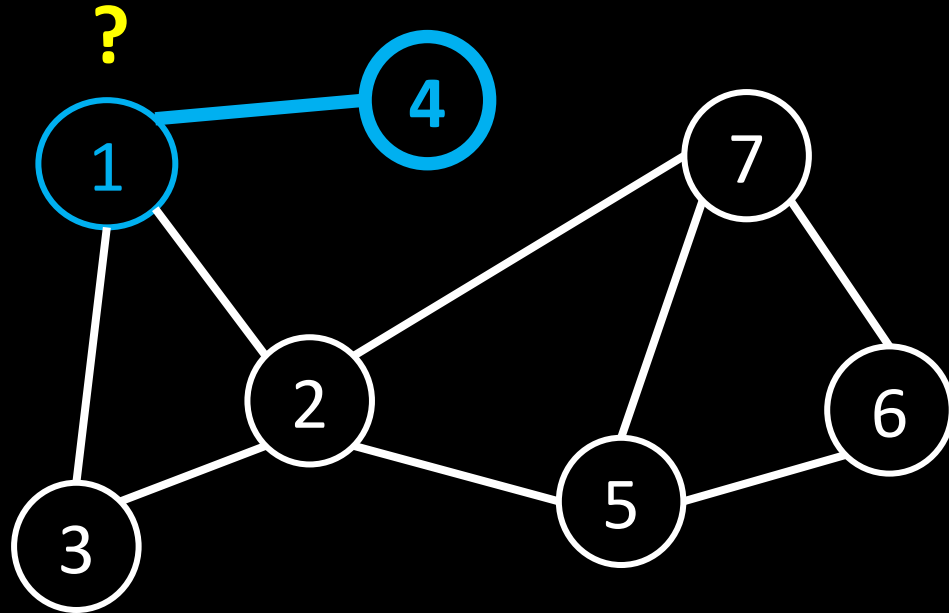


```
void dfs(int v) {  
    vis[v] = true;  
    for (int u : g[v])  
        if (!vis[u])  
            dfs(u);  
}
```

Adjacency list:

1: 4 2 3 dfs(1)
2: 1 3 5 7
3: 2 1
4: 1
5: 2 7 6
6: 7 5
7: 2 5 6

Depth-first search



1 4

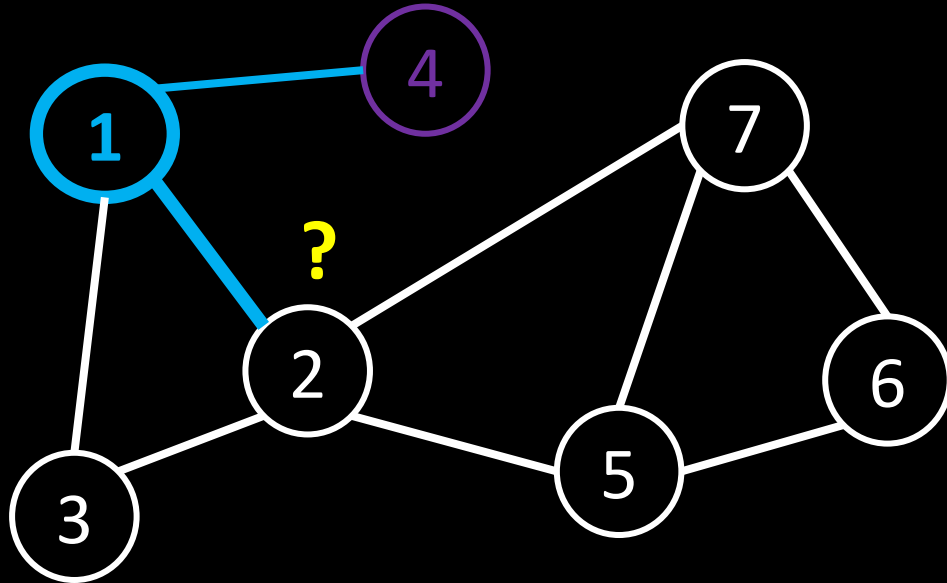
```
void dfs(int v) {  
    vis[v] = true;  
    for (int u : g[v])  
        if (!vis[u])  
            dfs(u);  
}
```

Adjacency list:

1: 4 2 3
2: 1 3 5 7
3: 2 1
4: 1
5: 2 7 6
6: 7 5
7: 2 5 6

dfs(1)
dfs(4)

Depth-first search



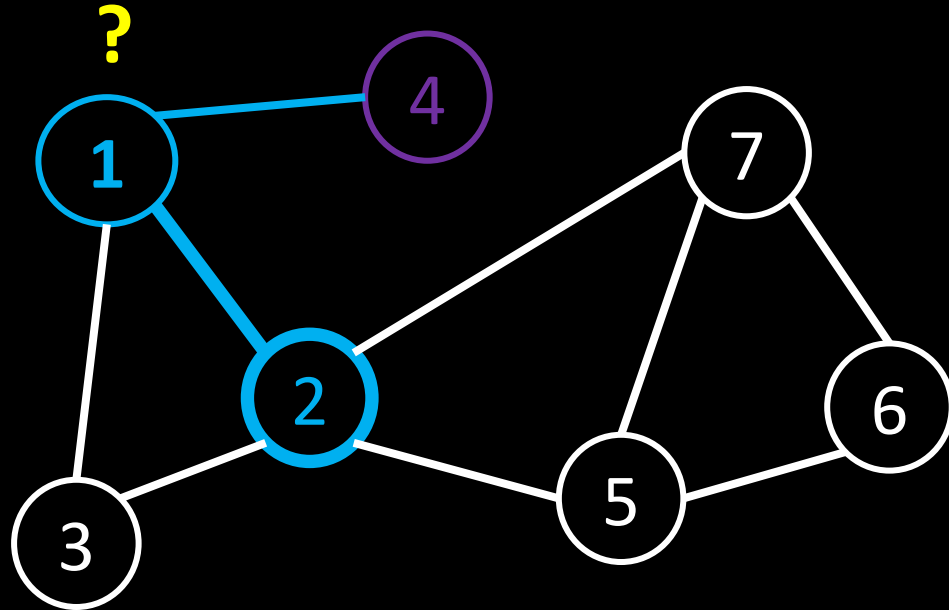
1
4

```
void dfs(int v) {  
    vis[v] = true;  
    for (int u : g[v])  
        if (!vis[u])  
            dfs(u);  
}
```

Adjacency list:

<u>1</u> :	4	2	3	dfs(1)
2:	1	3	5	7
3:	2	1		
4:	1			
5:	2	7	6	
6:	7	5		
7:	2	5	6	

Depth-first search



1 2
4

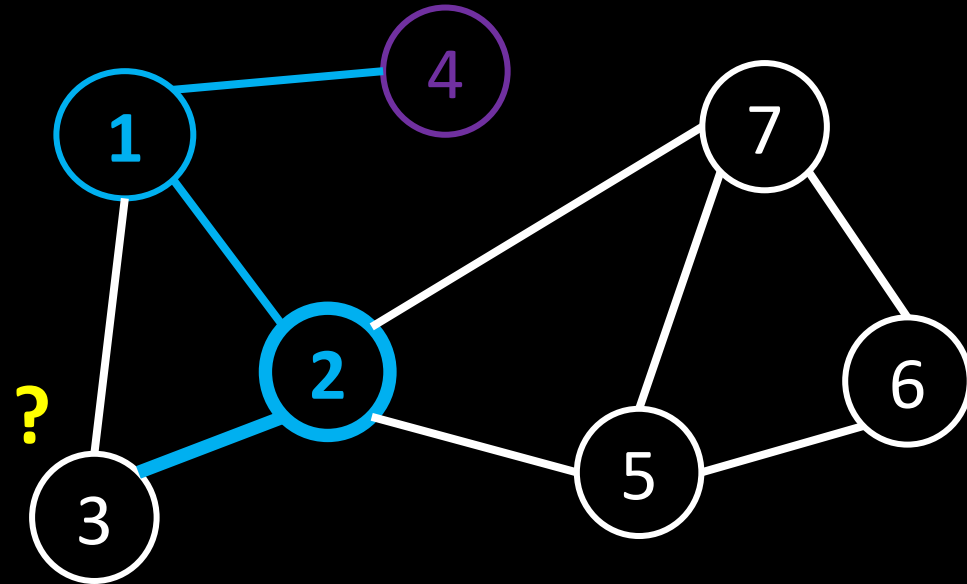
```
void dfs(int v) {  
    vis[v] = true;  
    for (int u : g[v])  
        if (!vis[u])  
            dfs(u);  
}
```

Adjacency list:

1: 4 2 3
2: 1 3 5 7
3: 2 1
4: 1
5: 2 7 6
6: 7 5
7: 2 5 6

dfs(1)
dfs(2)

Depth-first search



1 2
4

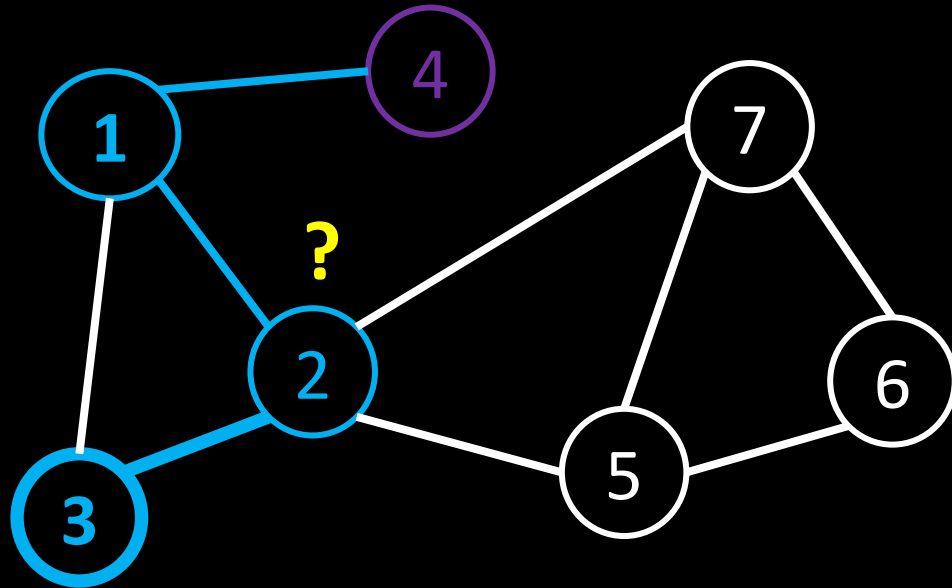
```
void dfs(int v) {  
    vis[v] = true;  
    for (int u : g[v])  
        if (!vis[u])  
            dfs(u);  
}
```

Adjacency list:

1: 4 2 3
2: 1 3 5 7
3: 2 1
4: 1
5: 2 7 6
6: 7 5
7: 2 5 6

dfs(1)
dfs(2)

Depth-first search



1 2 3
4

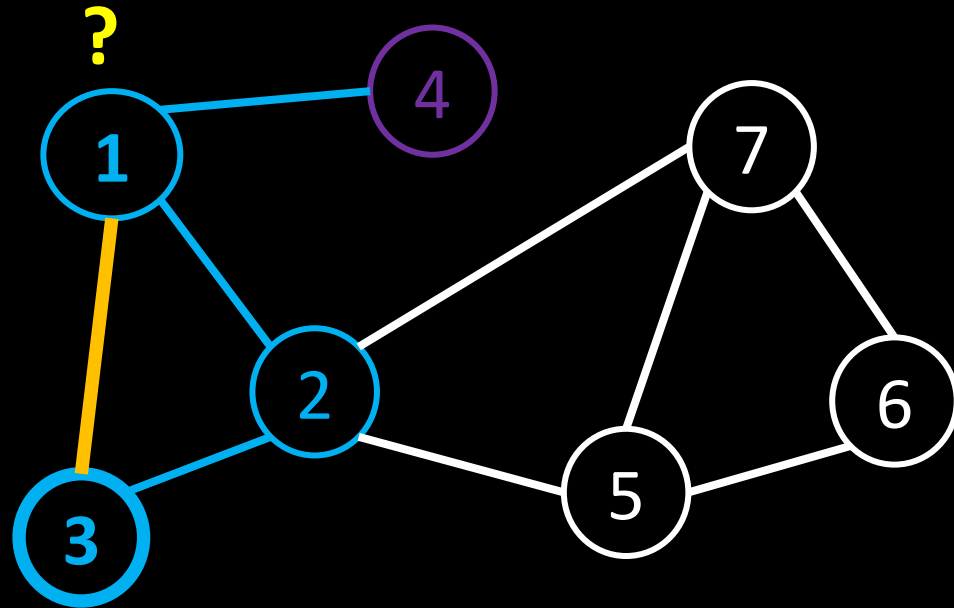
```
void dfs(int v) {  
    vis[v] = true;  
    for (int u : g[v])  
        if (!vis[u])  
            dfs(u);  
}
```

Adjacency list:

```
1: 4 2 3  
2: 1 3 5 7  
3: 2 1  
4: 1  
5: 2 7 6  
6: 7 5  
7: 2 5 6
```

dfs(1)
 dfs(2)
 dfs(3)

Depth-first search



1 2 3
4

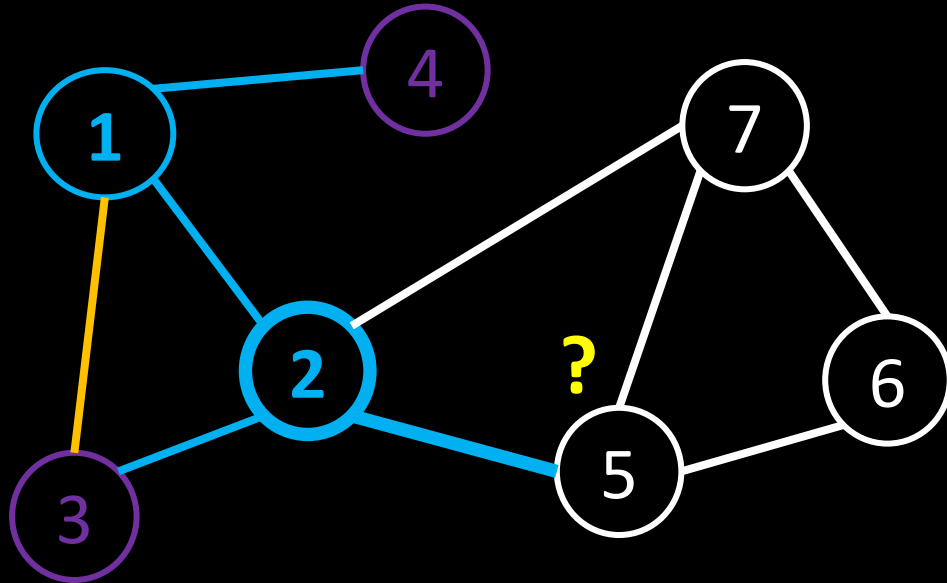
```
void dfs(int v) {  
    vis[v] = true;  
    for (int u : g[v])  
        if (!vis[u])  
            dfs(u);  
}
```

Adjacency list:

1: 4 2 3
2: 1 3 5 7
3: 2 1
4: 1
5: 2 7 6
6: 7 5
7: 2 5 6

dfs(1)
 dfs(2)
 dfs(3)

Depth-first search



1 2
4 3

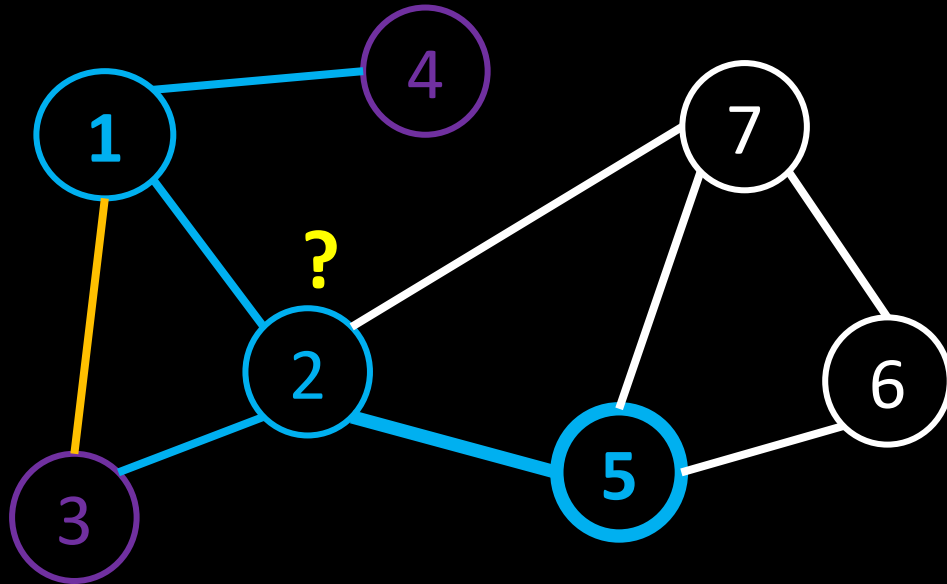
```
void dfs(int v) {  
    vis[v] = true;  
    for (int u : g[v])  
        if (!vis[u])  
            dfs(u);  
}
```

Adjacency list:

1: 4 2 3
2: 1 3 5 7
3: 2 1
4: 1
5: 2 7 6
6: 7 5
7: 2 5 6

dfs(1)
dfs(2)

Depth-first search



1 2 5
4 3

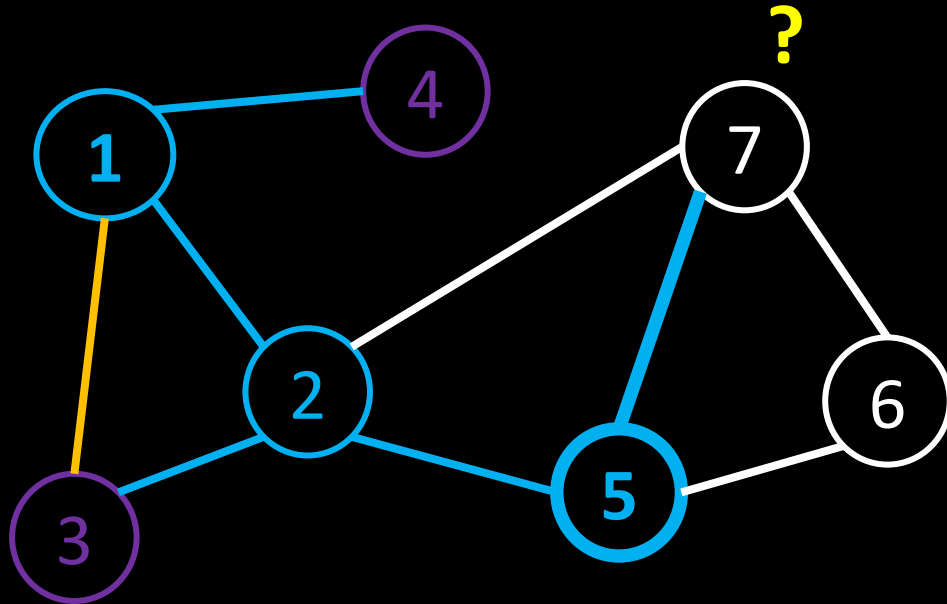
```
void dfs(int v) {  
    vis[v] = true;  
    for (int u : g[v])  
        if (!vis[u])  
            dfs(u);  
}
```

Adjacency list:

1: 4 2 3
2: 1 3 5 7
3: 2 1
4: 1
5: 2 7 6
6: 7 5
7: 2 5 6

dfs(1)
dfs(2)
dfs(5)

Depth-first search



1 2 5
4 3

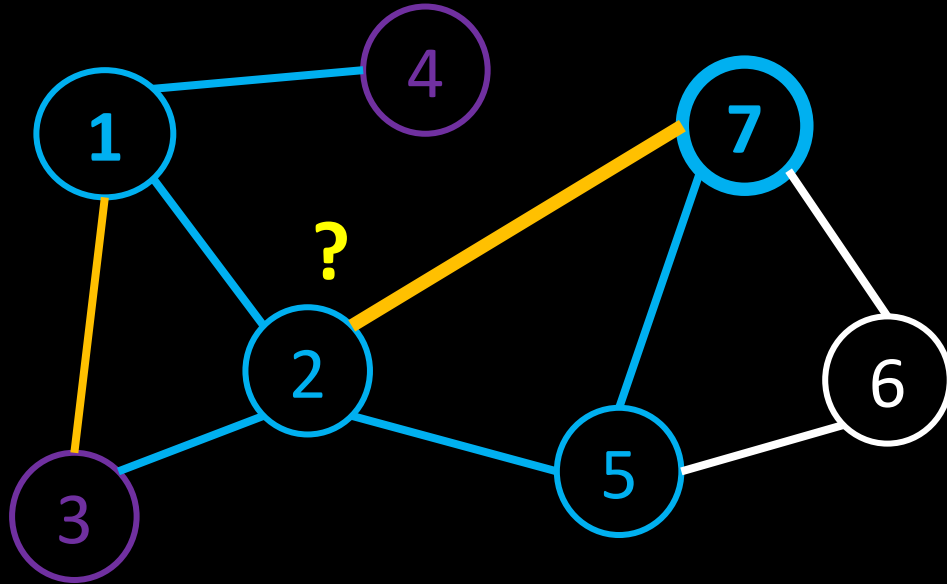
```
void dfs(int v) {  
    vis[v] = true;  
    for (int u : g[v])  
        if (!vis[u])  
            dfs(u);  
}
```

Adjacency list:

1: 4 2 3
2: 1 3 5 7
3: 2 1
4: 1
5: 2 7 6
6: 7 5
7: 2 5 6

dfs(1)
 dfs(2)
 dfs(5)

Depth-first search



1 2 5 7
4 3

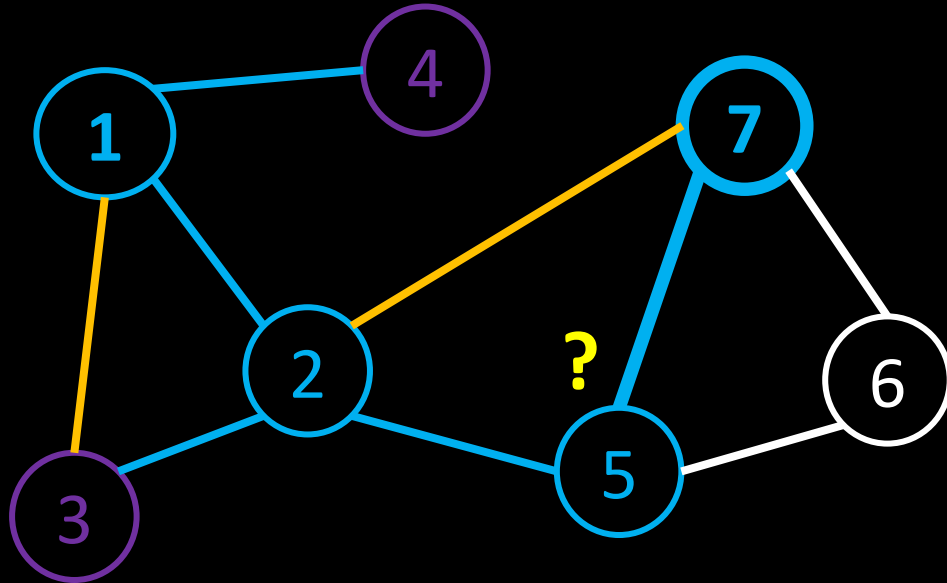
```
void dfs(int v) {  
    vis[v] = true;  
    for (int u : g[v])  
        if (!vis[u])  
            dfs(u);  
}
```

Adjacency list:

1: 4 2 3
2: 1 3 5 7
3: 2 1
4: 1
5: 2 7 6
6: 7 5
7: 2 5 6

dfs(1)
 dfs(2)
 dfs(5)
 dfs(7)

Depth-first search



1 2 5 7
4 3

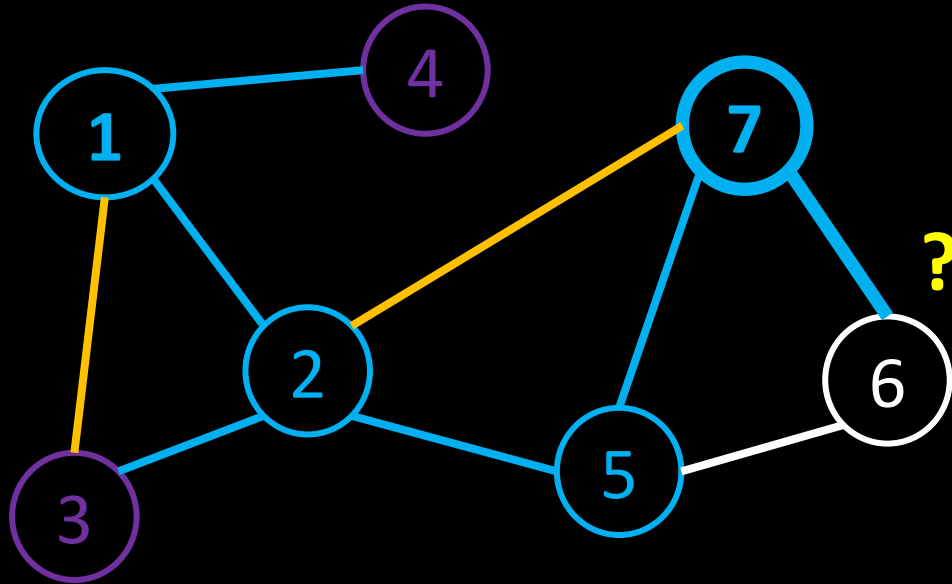
```
void dfs(int v) {  
    vis[v] = true;  
    for (int u : g[v])  
        if (!vis[u])  
            dfs(u);  
}
```

Adjacency list:

1: 4 2 3
2: 1 3 5 7
3: 2 1
4: 1
5: 2 7 6
6: 7 5
7: 2 5 6

dfs(1)
 dfs(2)
 dfs(5)
 dfs(7)

Depth-first search



1 2 5 7
4 3

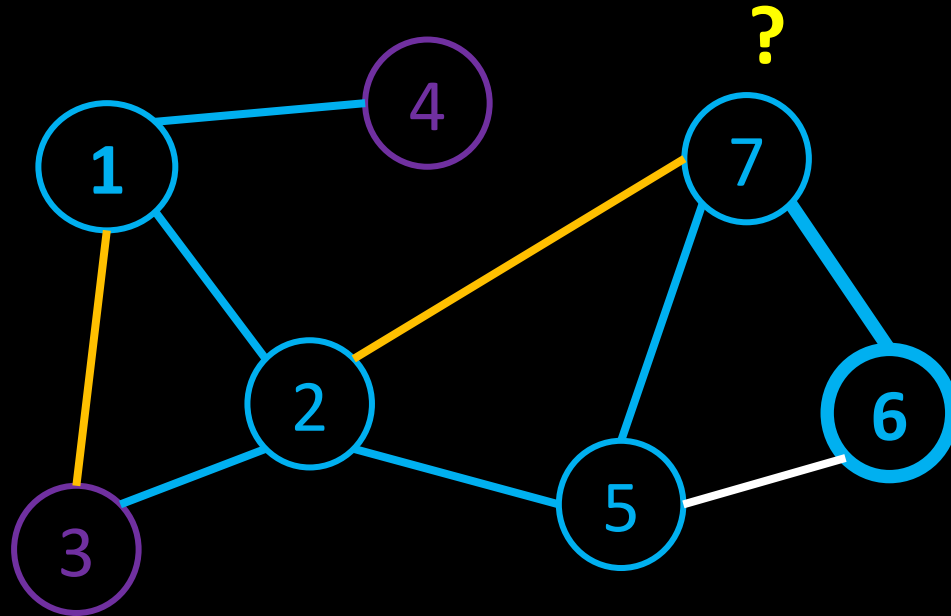
```
void dfs(int v) {  
    vis[v] = true;  
    for (int u : g[v])  
        if (!vis[u])  
            dfs(u);  
}
```

Adjacency list:

```
1: 4 2 3  
2: 1 3 5 7  
3: 2 1  
4: 1  
5: 2 7 6  
6: 7 5  
7: 2 5 6
```

```
dfs(1)  
  dfs(2)  
    dfs(5)  
      dfs(7)
```

Depth-first search



1 2 5 7 6
4 3

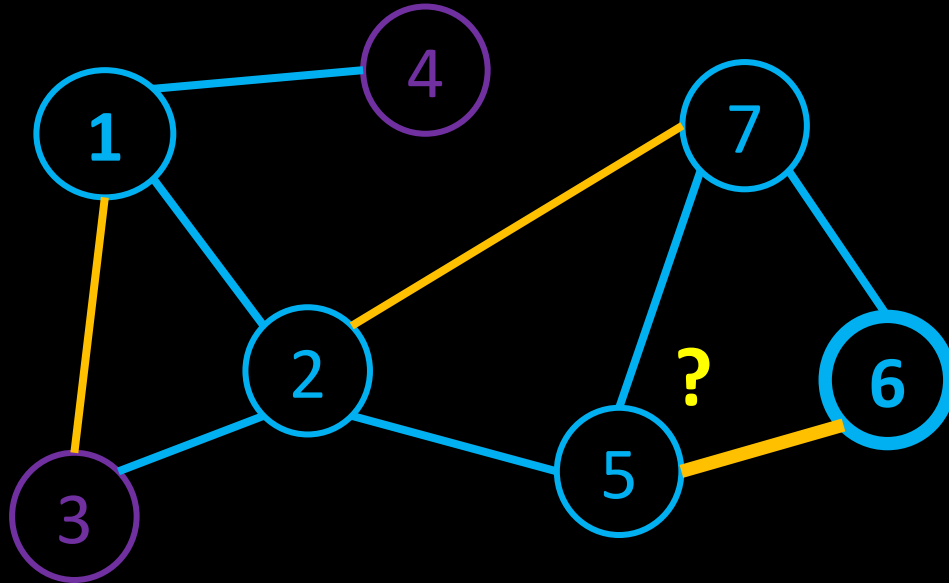
```
void dfs(int v) {  
    vis[v] = true;  
    for (int u : g[v])  
        if (!vis[u])  
            dfs(u);  
}
```

Adjacency list:

1: 4 2 3
2: 1 3 5 7
3: 2 1
4: 1
5: 2 7 6
6: 7 5
7: 2 5 6

dfs(1)
 dfs(2)
 dfs(5)
 dfs(7)
 dfs(6)

Depth-first search



1 2 5 7 6
4 3

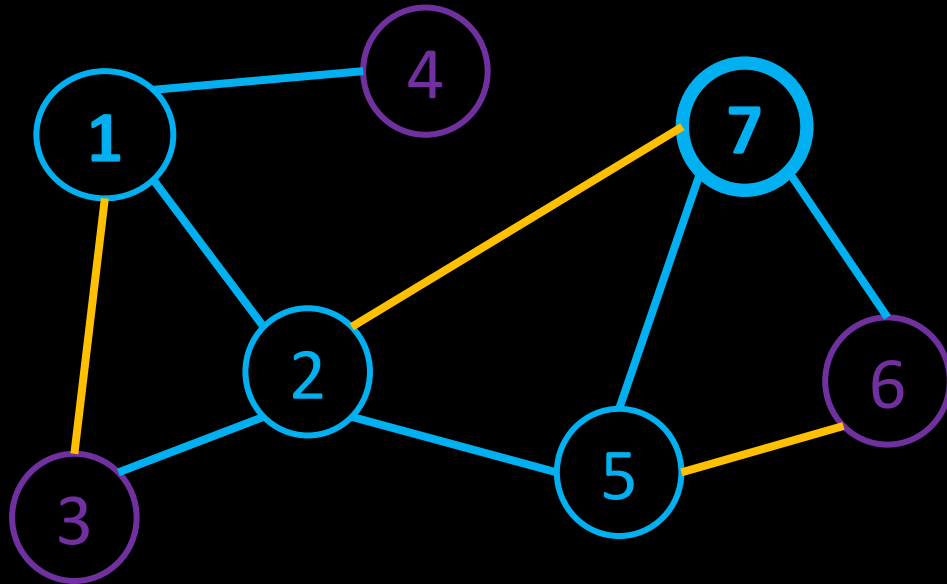
```
void dfs(int v) {  
    vis[v] = true;  
    for (int u : g[v])  
        if (!vis[u])  
            dfs(u);  
}
```

Adjacency list:

```
1: 4 2 3  
2: 1 3 5 7  
3: 2 1  
4: 1  
5: 2 7 6  
6: 7 5  
7: 2 5 6
```

```
dfs(1)  
  dfs(2)  
    dfs(5)  
      dfs(7)  
        dfs(6)
```

Depth-first search



1 2 5 7
4 3 6

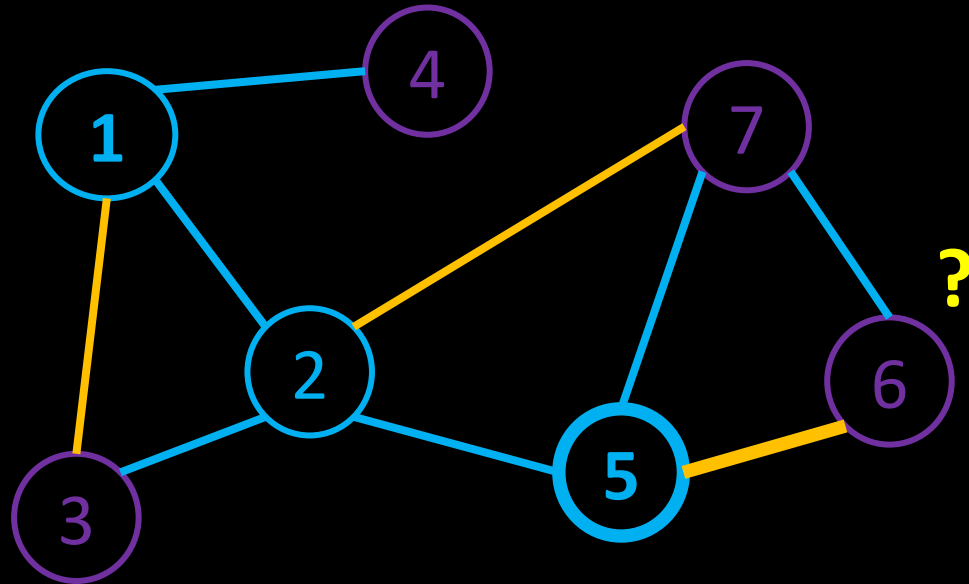
```
void dfs(int v) {  
    vis[v] = true;  
    for (int u : g[v])  
        if (!vis[u])  
            dfs(u);  
}
```

Adjacency list:

1: 4 2 3
2: 1 3 5 7
3: 2 1
4: 1
5: 2 7 6
6: 7 5
7: 2 5 6

dfs(1)
 dfs(2)
 dfs(5)
 dfs(7)

Depth-first search



1 2 5
4 3 6 7

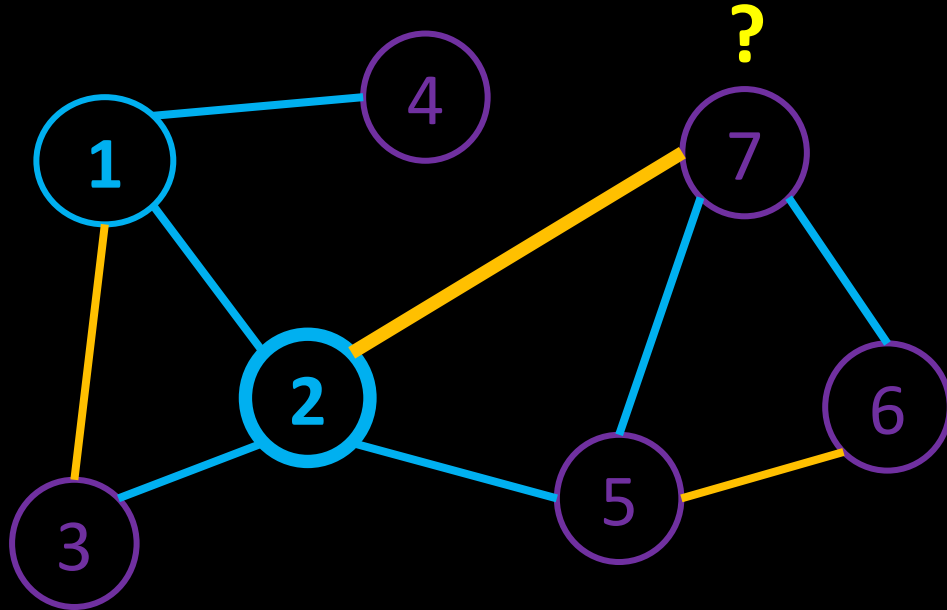
```
void dfs(int v) {  
    vis[v] = true;  
    for (int u : g[v])  
        if (!vis[u])  
            dfs(u);  
}
```

Szomszédsági lista

1: 4 2 3
2: 1 3 5 7
3: 2 1
4: 1
5: 2 7 6
6: 7 5
7: 2 5 6

dfs(1)
 dfs(2)
 dfs(5)

Depth-first search



1 2
4 3 6 7 5

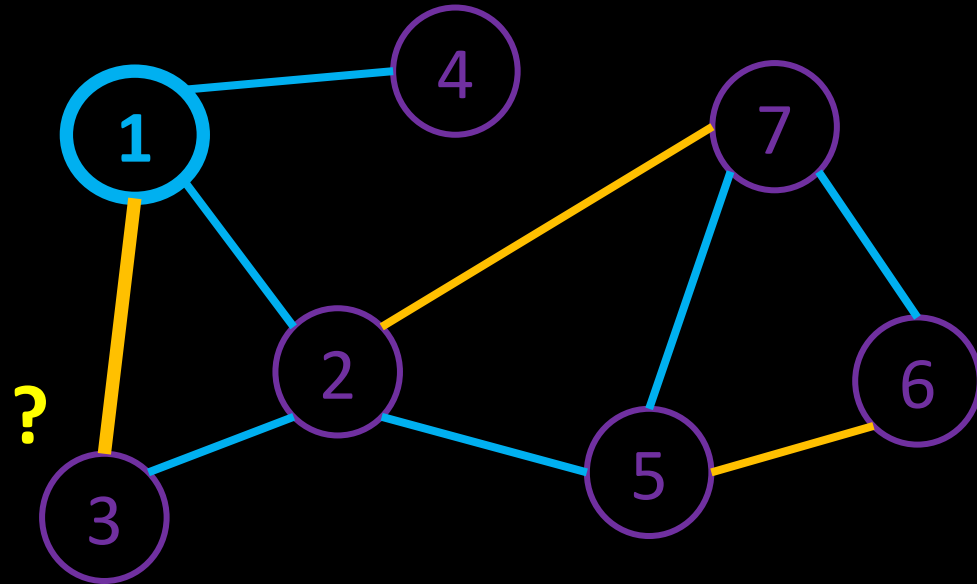
```
void dfs(int v) {  
    vis[v] = true;  
    for (int u : g[v])  
        if (!vis[u])  
            dfs(u);  
}
```

Adjacency list:

1: 4 2 3
2: 1 3 5 7
3: 2 1
4: 1
5: 2 7 6
6: 7 5
7: 2 5 6

dfs(1)
dfs(2)

Depth-first search



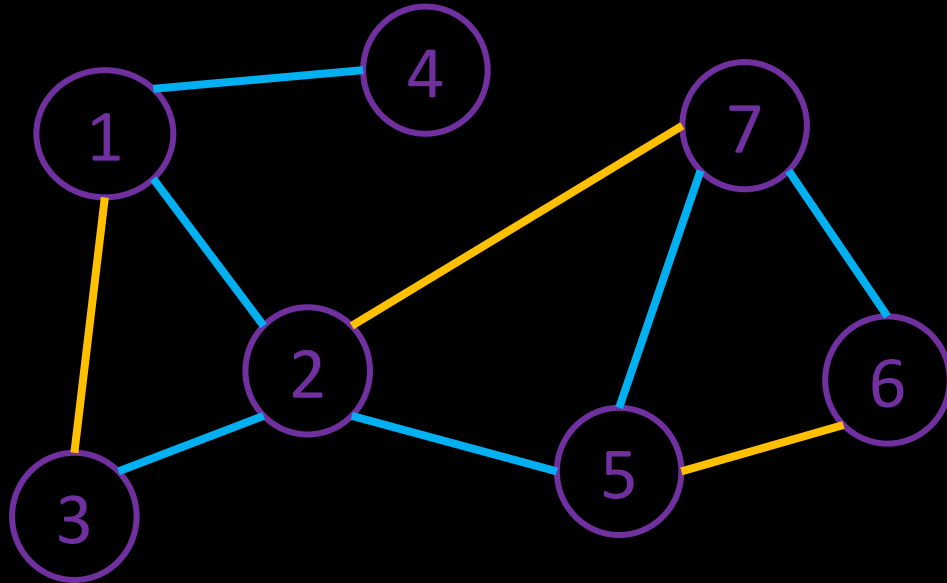
1
4 3 6 7 5 2

```
void dfs(int v) {  
    vis[v] = true;  
    for (int u : g[v])  
        if (!vis[u])  
            dfs(u);  
}
```

Adjacency list:

<u>1</u> :	4	2	3	dfs(1)
2:	1	3	5	7
3:	2	1		
4:	1			
5:	2	7	6	
6:	7	5		
7:	2	5	6	

Depth-first search



DSF in

1 4 2 3 5 7 6

DFS out

4 3 6 7 5 2 1

```
void dfs(int v) {  
    vis[v] = true;  
    for (int u : g[v])  
        if (!vis[u])  
            dfs(u);  
}
```

Adjacency list:

1: 4 2 3

2: 1 3 5 7

3: 2 1

4: 1

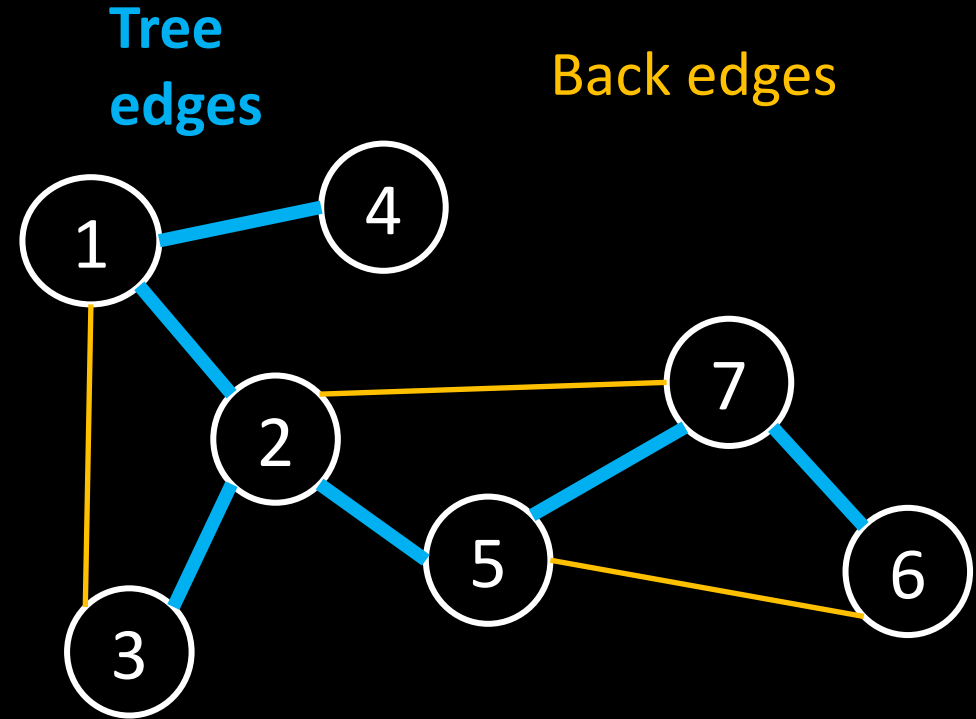
5: 2 7 6

6: 7 5

7: 2 5 6

DFS - properties

- The traversed edges create a tree
- Once we leave a vertex, we will have traversed every vertex reachable from it
- In case of undirected graphs: only tree edges and back edges
 - *In case of directed graphs: cross edges and forward edges as well*
- Time complexity: $O(N + E)$



Now let's see some exercises!